

SAPIENZA
UNIVERSITÀ DI ROMA

"SAPIENZA" UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

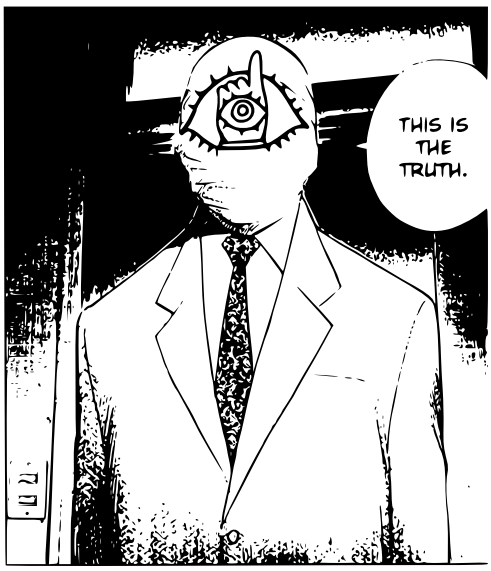
Autonomous And Mobile Robotics

Lecture notes

Author
Marco Casu
https://github.com/CasuFrost

October 28, 2026

This document summarizes and presents the topics for the Autonomous And Mobile Roboticscourse for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

1 Wheeled Mobile Robots **3**
1.1 Configuration Space . . . 3
1.1.1 Topology of a Space . . . 4
1.1.2 Distance in the Configuration Space . . . 6
1.2 Mechanics of Mobile Robots . . . 8
1.2.1 Main Kinematic Structures . . . 9
1.3 Constraints and Mobility . . . 9
1.3.1 Geometric Constraints . . . 10
1.3.2 Kinematic Constraints . . . 11
1.3.3 Non-Holonomic Constraints . . . 12
1.4 Kinematic Models . . . 14
1.4.1 The Unicycle Model . . . 16
1.4.2 The Bicycle Model . . . 17
1.5 Nonlinear Controllability . . . 21
1.5.1 The Unicycle Case . . . 24
1.5.2 The Bicycle Case . . . 24
1.5.3 Chained-Form . . . 25
1.6 Path and Trajectory Planning . . . 27
1.6.1 Flat Outputs . . . 29
1.6.2 Path Planning for the Unicycle . . . 31
1.6.3 Path Planning for a Chained-Form System . . . 32
1.6.4 Path Planning via Parametrized Inputs . . . 32
1.6.5 Choice of the Timing Law . . . 33

2 Motion Control **35**
2.1 Trajectory Tracking . . . 36
2.1.1 Approximate Linearization . . . 37
2.1.2 Output Error Feedback . . . 39
2.1.3 Static Feedback . . . 39
2.1.4 Dynamic Feedback . . . 40
2.1.5 Comparison of the Controllers . . . 41

CHAPTER

1

WHEELED MOBILE ROBOTS

1.1 Configuration Space

We can define a robot as a collection of n_b bodies moving in an operative space, denoted W and called the **workspace**. W can be $\mathbb{R}^2$ (if the robot moves on a plane, like a planar 2R) as well as $\mathbb{R}^3$, if the robot moves in the space. For example, an elbow-type spatial manipulator is a system of $n_b = 3$ rigid bodies moving in $W = \mathbb{R}^3$. A quadrotor is a single body ($n_b = 1$) moving in the same workspace $W = \mathbb{R}^3$.

To describe the complete pose of a planar 2R manipulator, we might use the position vector and roll-pitch-yaw angles for the two bodies, which requires a total of 12 parameters for the whole system, despite that, we know that 2 variables are enough to describe the whole configuration of the system.

Definition 1.1.1 : Configuration

The configuration of a robot is a **minimum** set of parameters that allows one to specify the posture[a] of the robot in the workspace.

[a] bt posture we mean the position of all the points of the robot.

The configuration of a robot can be defined in several ways, and a choice of a set of parameters is not unique, however, each robot always has a *minimum number* of parameters. We usually denote $\mathbf{q}$ the configuration vector:

$$\mathbf{q} = \begin{pmatrix} q_1 \\ \vdots \\ q_n \end{pmatrix} \tag{1.1}$$

in our context, it usually describes the Cartesian coordinates of a single body robot, or the angular coordinates of the body parts. The Cartesian coordinates have values in a Euclidean space, as $\mathbb{R}^2$ or $\mathbb{R}^3$, while the angular coordinates take values in a non-Euclidean space, such as the special orthogonal group $SO(3)$, the group of all the orthogonal 3×3 matrices with determinant $+1$, this group describes the orientation of a body in a 3D Euclidean space.

Definition 1.1.2 : Configuration Space

The configuration space, usually denoted $\mathcal{C}$, is the set of all possible values of the configuration of a robot. The *dimension* of $\mathcal{C}$ is the number n of parameters (generalized coordinates) in $\mathbf{q}$.

Let's see an example: consider a disk robot that moves on a plane, that can also rotate about its vertical axis to point towards a desired direction, shown in Figure 1.1.

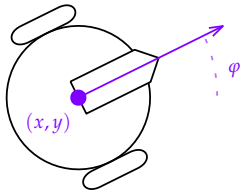


Figure 1.1: Disk robot with a tick in $\mathbb{R}^2$.

Its configuration is given by two Cartesian coordinates (x, y) and a value $\varphi \in S^1$ representing the angle between the tick and the horizontal axis. Hence, its configuration space is:

$$\mathcal{C} = \mathbb{R}^2 \times SO(2) = SE(2) \tag{1.2}$$

and it's not Euclidean, despite being called *special Euclidean group*. A polyhedral robot that can freely move in space is described by three Cartesian coordinates (x, y, z) and three roll-pitch-yaw angles (α, β, γ) describing its orientation, hence the configuration space is $SE(3)$:

$$\mathcal{C} = \mathbb{R}^3 \times SO(3) = SE(3). \tag{1.3}$$

We have seen in previous courses of robotics that the number of generalized coordinates of an n-link serial manipulator is exactly n.

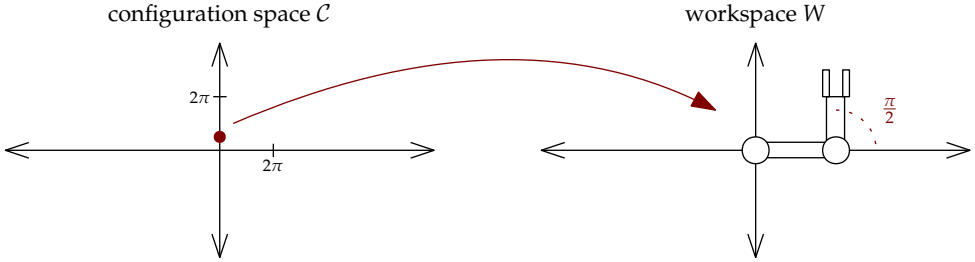
1.1.1 Topology of a Space

We are interested in describing the configuration space of a given manipulator, since it's not Euclidean (in most cases), we would like to have an idea of how to navigate this space. Before defining what a *manifold* is, we give some examples.

A 2R planar manipulator can be described by two angles, hence two values in $SO(2)$, so its configuration space is

$$\mathcal{C} = SO(2) \times SO(2) = SO(2)^2 \tag{1.4}$$

Let's try to represent graphically $\mathcal{C}$. We might represent a configuration as a point in the Cartesian plane:



The problem of this representation is that different configurations $\mathbf{q} \in \mathcal{C}$ map to the same posture of the robot in W, as shown in Figure 1.2.

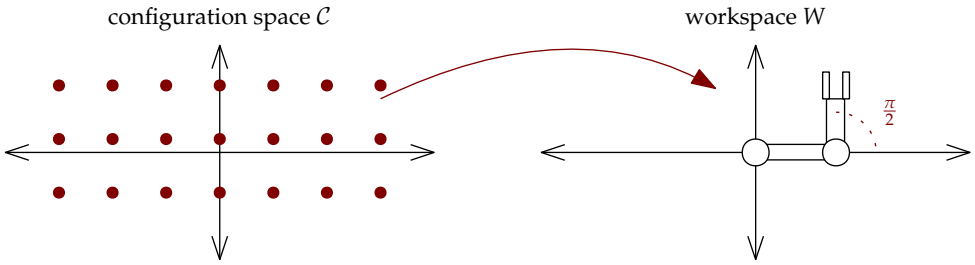
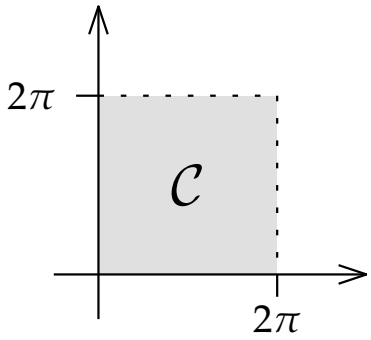
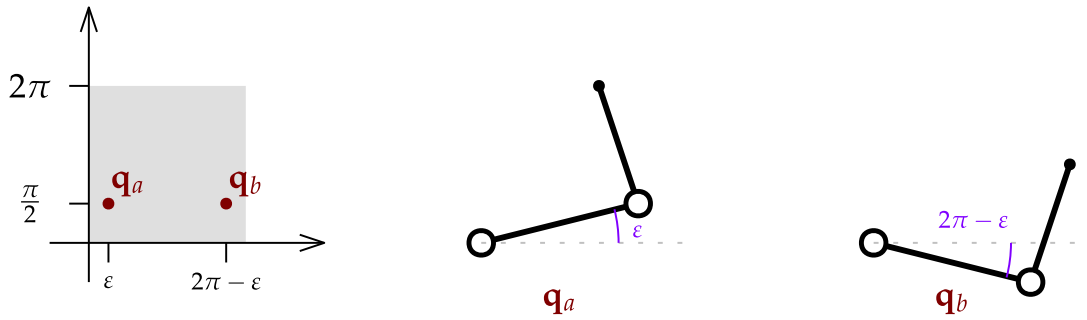


Figure 1.2: Different configurations map to the same posture.

To overcome this problem, we might try to represent the configuration space as a square in the plane $[0, 2\pi) \times [0, 2\pi)$:



This representation also presents a problem: The distance between two configurations in $\mathcal{C}$ does not represent the true distance between two postures in the workspace: Two very distant configurations $\mathbf{q}_a$ and $\mathbf{q}_b$ in $\mathcal{C}$ can be mapped to two very close postures in W:



This representation is said to be **not topologically correct**. To give a correct representation of $\mathcal{C}$ we need to introduce the concept of **manifold**. We will not give a formal and complete definition (as given in topology or differential geometry courses).

Definition 1.1.3 : Manifold (loose definition)

A **manifold** is a topological space that is locally homeomorphic to $\mathbb{R}^n$.

As we should know from previous robotics courses, the configuration space of a 2R is a torus, a manifold that is locally homeomorphic to $\mathbb{R}^2$, and can be visualized in the 3D space through an embedding as shown in Figure 1.3.

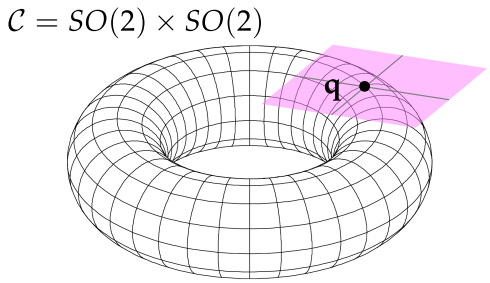


Figure 1.3: 3D representation of a torus.

1.1.2 Distance in the Configuration Space

Definition 1.1.4 : Distance in the Configuration Space

Given a configuration space $\mathcal{C}$, the **distance** is a function $d : \mathcal{C} \times \mathcal{C} \to \mathbb{R}$ such that, for each $\mathbf{q}_a, \mathbf{q}_b, \mathbf{q}_c$ in $\mathcal{C}$ we have:

$$d(\mathbf{q}_a, \mathbf{q}_b) \geq 0 \tag{1.5}$$
$$d(\mathbf{q}_a, \mathbf{q}_b) = 0 \iff \mathbf{q}_a = \mathbf{q}_b \tag{1.6}$$
$$d(\mathbf{q}_a, \mathbf{q}_b) = d(\mathbf{q}_b, \mathbf{q}_a) \tag{1.7}$$
$$d(\mathbf{q}_a, \mathbf{q}_b) \leq d(\mathbf{q}_a, \mathbf{q}_c) + d(\mathbf{q}_c, \mathbf{q}_b). \tag{1.8}$$

We know that the distance between two vectors $\mathbf{u}, \mathbf{v}$ in $\mathbb{R}^n$ is given by $\|\mathbf{u} - \mathbf{v}\|$. The distance between two points is the length of the *shortest path* between them, for a general manifold, this path is not always a straight line as for the Euclidean space.

Definition 1.1.5 : Geodesic

For a general manifold, the shortest path between two points is called **geodesic**, and the distance between the points is the length of their geodesic.

For example, consider the shortest path between two points on the surface of a sphere (a 2-manifold): the geodesic lies on the great circle (the circle with the biggest radius that passes through the two points), as shown in Figure 1.4. In general, for an arbitrary manifold, it is hard to determine the geodesic between two points.

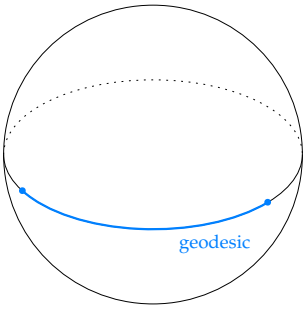


Figure 1.4: Geodesic on a sphere.

In the robotics context, we want to define the distance between two postures of a robot, this distance is defined in the workspace W, and to define that we don't need to specify directly the distance between their respective configurations in $\mathcal{C}$ (it would be harder to find the geodesic on $\mathcal{C}$).

Given a robot and its current configuration $\mathbf{q}$, we can define two functions:

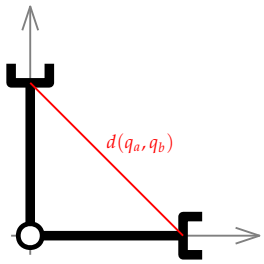
- $B(\mathbf{q})$ is a function that assigns to $\mathbf{q}$ the volume (exact region) of the workspace that the robot occupies when it is in the configuration $\mathbf{q}$, so it assigns to each configuration $\mathbf{q} \in \mathcal{C}$ a set of points in $\mathbb{R}^3$.
- $\mathbf{p}(\mathbf{q})$ is a function that assigns to $\mathbf{q}$ the exact position of a certain point of the robot when it's in the configuration $\mathbf{q}$.

For example, we might let $\mathbf{p}$ be the center of mass of the manipulator, and observe how it changes by varying $\mathbf{q} \in \mathcal{C}$. One simple way to define a distance metric is to consider the maximum Euclidean distance between two points of the manipulator in two different configurations:

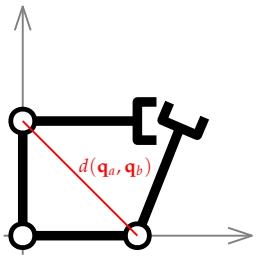
$$d(\mathbf{q}_a, \mathbf{q}_b) = \max_{\mathbf{p} \in \text{robot}} \|\mathbf{p}(\mathbf{q}_a) - \mathbf{p}(\mathbf{q}_b)\| \tag{1.9}$$

This distance is called **displacement metric**. Notice how in the definition we are writing $\mathbf{p} \in$ robot, by letting robot be the set of all the points in the structure of the manipulator, which clearly depend on the current configuration. This kind of metric works under the assumption that, if the region occupied by the robot is similar in two different postures, then the distance between their respective configurations is small:

$$d(\mathbf{q}_a, \mathbf{q}_b) \simeq 0 \implies B(\mathbf{q}_a) \simeq B(\mathbf{q}_b). \tag{1.10}$$



1R manipulator



2R manipulator

To compute the distance in Equation (1.9), we should check each point of the robot, and in general this is an uncountably infinite set. If the links are polyhedral shapes, the problem becomes finite, since it is possible to find the maximum distance between any pair of polytopes in the three dimensional space.

A more computationally feasible approach is to check only for a finite set of points in the robot structure, called **control points**. Let

$$P = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_c\} \tag{1.11}$$

be a set of c points of the robot (in general, they all depend on the configuration $\mathbf{q}$). We compute the distance between two postures in the following way:

$$d(\mathbf{q}_a, \mathbf{q}_b) = \max_{1 \le i \le c} \|\mathbf{p}_i(\mathbf{q}_a) - \mathbf{p}_i(\mathbf{q}_b)\|. \tag{1.12}$$

Not all the points of a robot are feasible control points, for example, the base of a manipulator usually doesn't change with the configuration. P is a set of *proper control points* if the function d as defined in Equation (1.12) satisfies the properties of a distance given in Definition 1.1.4.

Observation 1.1.1 : Approximation for Local Motion

Since a smooth manifold is locally homeomorphic to $\mathbb{R}^n$, for sufficiently close configurations we can approximate the distance on the manifold (assuming it is smooth) with the Euclidean distance.

Observation 1.1.2 : Prismatic Joints

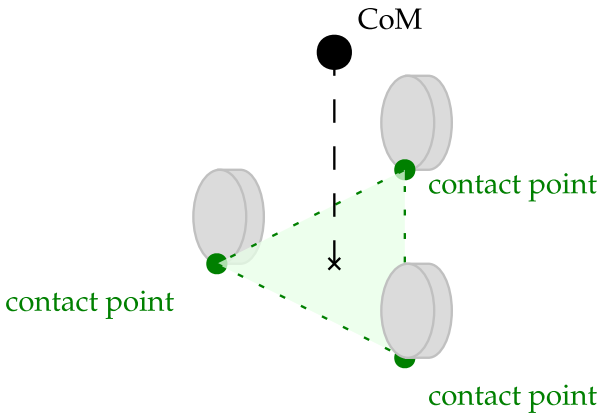
The configuration space of a robot with *only* prismatic joints is Euclidean.

1.2 Mechanics of Mobile Robots

The main sources of movement for a mobile robot are:

- Wheels: wheeled robots usually consist of one single rigid body, with the wheels in contact with the ground.
- Feet: legged robots, several rigid bodies articulated through joints.

The wheels are in contact with the ground, and this introduces some constraints. Each wheel defines a point of contact with the ground, we call the **support polygon** the convex hull of all the contact points. A wheeled robot is in **static balance** if the projection on the ground of its center of mass falls inside the support polygon. Of course a robot must be balanced in order not to fall to the ground.



The convex hull of two points is a line segment, hence a robot with only two wheels can't be easily balanced in a static fashion.

There are three main types of wheels:

- **Fixed Wheels**: They have a fixed orientation: Can roll but can't steer about their vertical axis (see Figure 1.5). They are usually actuated, if passive, they are used for balance and offer a contact point.
- **Steerable Wheels**: They can steer and are used for orientation, hence they are typically active (e.g. the front wheels of a car).
- **Caster Wheels**: They have a variable orientation but they are typically not actuated. Their axis of orientation are not aligned, this property makes the wheels align automatically with the direction of motion, hence they are "invisible" kinematically, but offer a contact point.

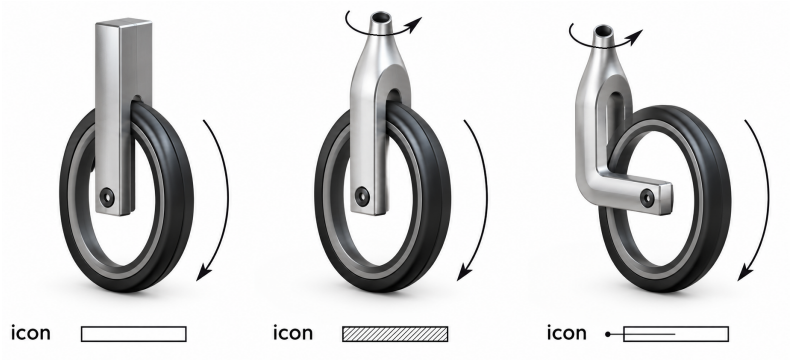
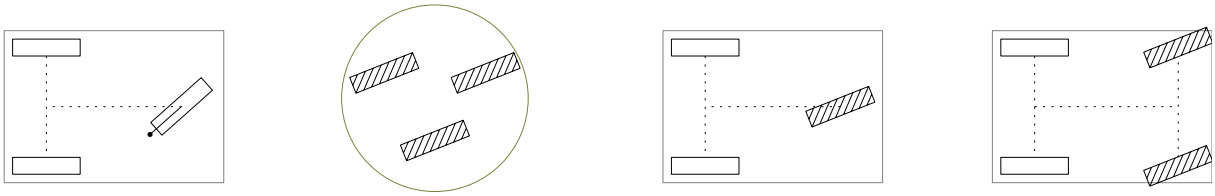


Figure 1.5: Fixed, steerable and caster wheels.

1.2.1 Main Kinematic Structures

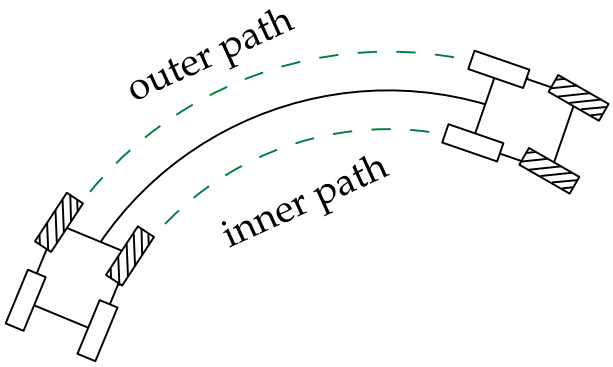
The simplest wheeled robot that we can have is the **differential drive** robot: it uses two aligned fixed wheels that are independently actuated, and one caster wheel added for balance. The **synchro-drive** mobile robots have three steerable wheels and two actuators: one controls simultaneously the orientation of all the wheels, and the other rolls them together. The base over the wheels hence will never change orientation. This type of configuration is preferable when the body to move is particularly heavy.



differential synchro-drive tricycle car-like

The tricycle has two fixed wheels in the back and one actuated steerable wheel in the front, the car-like is similar, but with two steerable wheels.

Consider the following path taken by a car-like robot:



In the same time interval, the wheels on the outer path travels a longer distance with respect to the wheels on the inner path, hence their rotation speed must be higher. In order to avoid using two distinct actuators for the front wheels we use a device called **differential**.

Definition 1.2.1 : Differential

A mechanical differential is a gear assembly (usually using bevel or planetary gears) connected to a single drive motor. It splits power to two wheels on the same axle while allowing them to rotate at different speeds. When a robot with a single motor turns, the outside wheel must travel a larger arc than the inside wheel. The differential lets the wheels spin at their own natural speeds, preventing tire slip, scrubbing, and mechanical strain on the drivetrain.

1.3 Constraints and Mobility

The presence of the wheels in contact with the ground introduces a series of *geometric* and *kinematic* constraints. We will also see the relationship between these two types of constraints and the implications in mobility.

1.3.1 Geometric Constraints

We have seen how the configuration $\mathbf{q}$ is in general a point of a manifold $\mathcal{C}$, that is locally homeomorphic to $\mathbb{R}^n$, with n being the dimensionality of the configuration space $\mathcal{C}$. We will focus on equality time-invariant constraints. All the geometric constraints are expressed in the following form:

$$h(\mathbf{q}) = 0 \tag{1.13}$$

and they depend only on the configuration. Usually we have $k < n$ geometric constraints, and they are reflected in the degrees of freedom of a robot:

$$h_1(\mathbf{q}) = 0 \tag{1.14}$$

$$\vdots \tag{1.15}$$

$$h_k(\mathbf{q}) = 0. \tag{1.16}$$

If the configuration space is a 2-manifold, one single geometric constraint will reduce the degrees of freedom of the robot, constraining it to move on a 1-dimensional manifold (a curve).

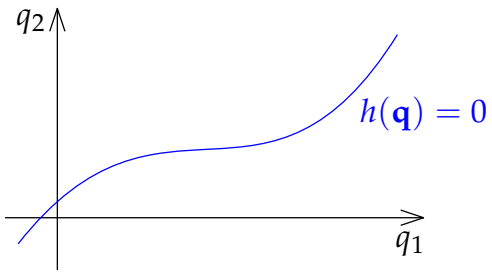


Figure 1.6: If the configuration space is $\mathbb{R}^2$, one constraint will let the configurations lie on a curve in the plane.

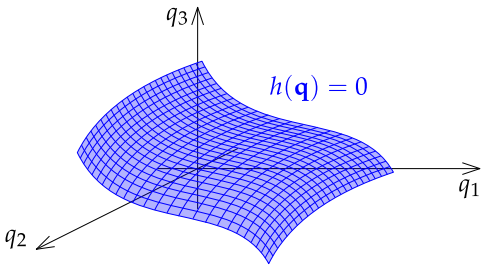


Figure 1.7: If the configuration space is $\mathbb{R}^3$, one constraint will let the configurations lie on a surface in the space.

In general, a robot with a configuration space that is an n-manifold with k independent geometric constraints, will move on an $(n-k)$-manifold. Instead of using the vector $\mathbf{q}$, we can reduce the number of coordinates in many ways, like for one constraint:

$$h(\mathbf{q}) = 0 \tag{1.17}$$

we can solve the equation for a coordinate q_i:

$$q_i = \tilde{h}(q_1, \ldots, q_{i-1}, q_{i+1}, \ldots, q_n). \tag{1.18}$$

For example, consider a 2 degrees of freedom robot with the following constraint:

$$q_1^2 + q_2^2 = 1 \tag{1.19}$$

the configuration space is limited to the unit circle in $\mathbb{R}^2$ (1-manifold), we solve the constraint for q_2:

$$q_2 = \pm\sqrt{1 - q_1^2}. \tag{1.20}$$

Actually this is not a good way to choose a coordinate, since the $\pm$ term prevents us from assigning a single value of q_2 to each value of q_1, requiring a local resolution. For this example of the circle it is better to use the arc length $s \in S^1$ as the single coordinate.

Observation 1.3.1 : Sub-Manifold

If the configuration space $\mathcal{C}$ is an n-manifold, the k independent constraints define a $(n-k)$-manifold embedded in $\mathcal{C}$, that we also call sub-manifold.

1.3.2 Kinematic Constraints

A kinematic constraint is local, it prevents the configuration from moving towards some directions. A general constraint of this kind is in the form:

$$a(\mathbf{q}, \dot{\mathbf{q}}) = 0 \tag{1.21}$$

it depends on the state of the robot. In every given configuration $\mathbf{q}^*$, we can immediately infer from $a(\mathbf{q}^*, \dot{\mathbf{q}}) = 0$ which velocities are allowed. All the relevant kinematic constraints in the robotic domain are in **Pfaffian form**:

$$\mathbf{a}^\top(\mathbf{q})\dot{\mathbf{q}} = 0 \tag{1.22}$$

with $\mathbf{a}$ being an n dimensional vector that depends on the configuration. The Pfaffian form expresses a condition of orthogonality: in a given configuration the velocity must be orthogonal to the vector $\mathbf{a}$. More kinematic constraints can be arranged in matrix form:

$$\begin{pmatrix} \mathbf{a}_1^\top(\mathbf{q}) \\ \mathbf{a}_2^\top(\mathbf{q}) \\ \vdots \\ \mathbf{a}_k^\top(\mathbf{q}) \end{pmatrix} \dot{\mathbf{q}} = \mathbf{0} \implies A(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \iff \dot{\mathbf{q}} \in \mathcal{N}(A). \tag{1.23}$$

The admissible velocities are the ones that lie in the null space of A[1], hence they lie in a linear space.

Observation 1.3.2 : Relationship Between Geometric and Kinematic Constraints

Any geometric constraint implies a kinematic constraint.

Consider a geometric constraint

$$h(\mathbf{q}) = 0 \tag{1.24}$$

this constraint must be satisfied as the time goes on, hence its derivative should also satisfy the following relation:

$$\dot{h}(\mathbf{q}) = 0 \tag{1.25}$$

by applying the chain rule:

$$\dot{h}(\mathbf{q}) = \frac{\partial h}{\partial \mathbf{q}}\dot{\mathbf{q}} = 0 \tag{1.26}$$

$\frac{\partial h}{\partial \mathbf{q}}$ is a row vector, the transpose of the gradient of h:

$$(\nabla_{\mathbf{q}} h)^\top \dot{\mathbf{q}} = 0 \tag{1.27}$$

this is exactly a kinematic constraint in Pfaffian form. The gradient of h evaluated on a point is the normal vector, i.e. orthogonal to the manifold on which the configuration is constrained to move, hence the resulting kinematic constraint is preventing the robot from going out of the sub-manifold defined by the geometric constraints.

[1] Sometimes, we let the dependencies of a term implicit.

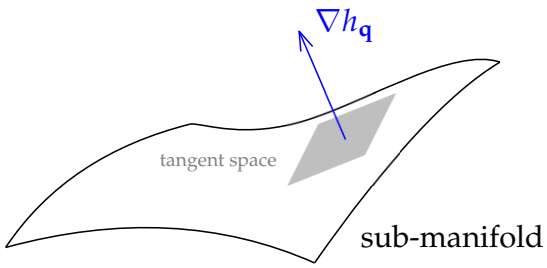


Figure 1.8: The admissible velocities are the ones that lie on the tangent space of the sub-manifold in the given configuration.

We have seen how a geometric constraint always implies a kinematic one. The opposite is not generally true, consider a generic kinematic constraint:

$$\mathbf{a}^\top(\mathbf{q})\dot{\mathbf{q}} = 0 \tag{1.28}$$

this implies a geometric constraint if there exists a function $h(\mathbf{q})$ such that

$$\nabla_{\mathbf{q}} h = \mathbf{a} \tag{1.29}$$

hence the constraint defined by $\mathbf{a}(\mathbf{q})$ should be *integrable*.

Definition 1.3.1 : Holonomic Constraint

An integrable kinematic constraint is called **holonomic**, and it defines a geometric constraint.

What conditions determine if a constraint is integrable or not? In general, the conditions are stated in the *Frobenius Theorem*, we will not discuss this result, which belongs to the domain of differential geometry. We will determine if a given constraint is holonomic or not by a different way.

Consider the following kinematic constraint for a 2-dimensional robot:

$$q_1\dot{q}_1 + q_2\dot{q}_2 = 0 \tag{1.30}$$

is this integrable? The answer is yes. The resulting geometric constraint is

$$q_1^2 + q_2^2 = c, \quad c > 0. \tag{1.31}$$

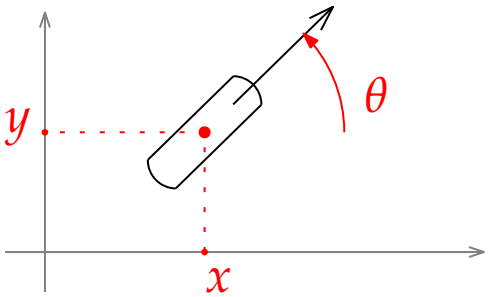
In general, in Equation (1.29) we can add a scalar integrating factor that depend on the configuration:

$$\frac{\partial h}{\partial \mathbf{q}} = \gamma(\mathbf{q})\mathbf{a}^\top(\mathbf{q}) \tag{1.32}$$

since the right-hand side is zero, γ can be dropped.

1.3.3 Non-Holonomic Constraints

A non-holonomic constraint defines a local limitation, it stops the robot from moving towards a certain direction when it is in that specific configuration, it does not prevent the robot from reaching a configuration by following an allowed path. The simplest example of a system with a non-holonomic constraint is a rolling disk.



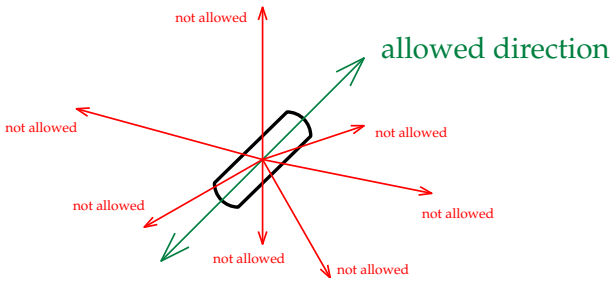
The configuration is given by the vector $\begin{pmatrix} x & y & \theta \end{pmatrix}^\top$, θ is its orientation with respect to the horizontal axis. It is clear that the disk in pure rolling (no slippage) can move *only* in the direction that is pointing (by rolling), hence can't move laterally. This constraint is expressed as follows:

$$\sin\theta\dot{x} - \cos\theta\dot{y} = 0 \tag{1.33}$$

or in vector form:

$$\begin{pmatrix} \sin\theta & -\cos\theta & 0 \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = 0 \tag{1.34}$$

$\mathbf{n}^\top = \begin{pmatrix} \sin\theta & -\cos\theta \end{pmatrix}$ is the vector *normal* to the direction. Any velocity that has a component on the line spanned by $\mathbf{n}$ is not allowed.



This constraint does not imply any global limitation, any configuration is still reachable, hence it is a non-holonomic constraint. To prove this, instead of using the Frobenius theorem, we give a non-formal proof by showing that, for any pair of configurations $\mathbf{q}_s$, $\mathbf{q}_g$, there exists a trajectory in the configuration space that connects these two configurations without violating the kinematic constraint. Let

$$\mathbf{q}_s = \begin{pmatrix} x_s \\ y_s \\ \theta_s \end{pmatrix}, \quad \mathbf{q}_g = \begin{pmatrix} x_g \\ y_g \\ \theta_g \end{pmatrix}. \tag{1.35}$$

1. $\dot{\theta}$ is not constrained, first of all we move the orientation from θ_s to θ^*, where θ^* is defined as the angle such that
$$\begin{pmatrix} \sin\theta^* & -\cos\theta^* \end{pmatrix} \begin{pmatrix} x_g - x_s \\ y_g - y_s \end{pmatrix} = 0. \tag{1.36}$$
2. Then, we can move on the direction $\begin{pmatrix} x_g - x_s & y_g - y_s \end{pmatrix}^\top$, in this way we can reach the desired position $\begin{pmatrix} x_g & y_g \end{pmatrix}^\top$.
3. At the end, we move the orientation from θ^* to θ_g.

This is called **universal maneuver**.

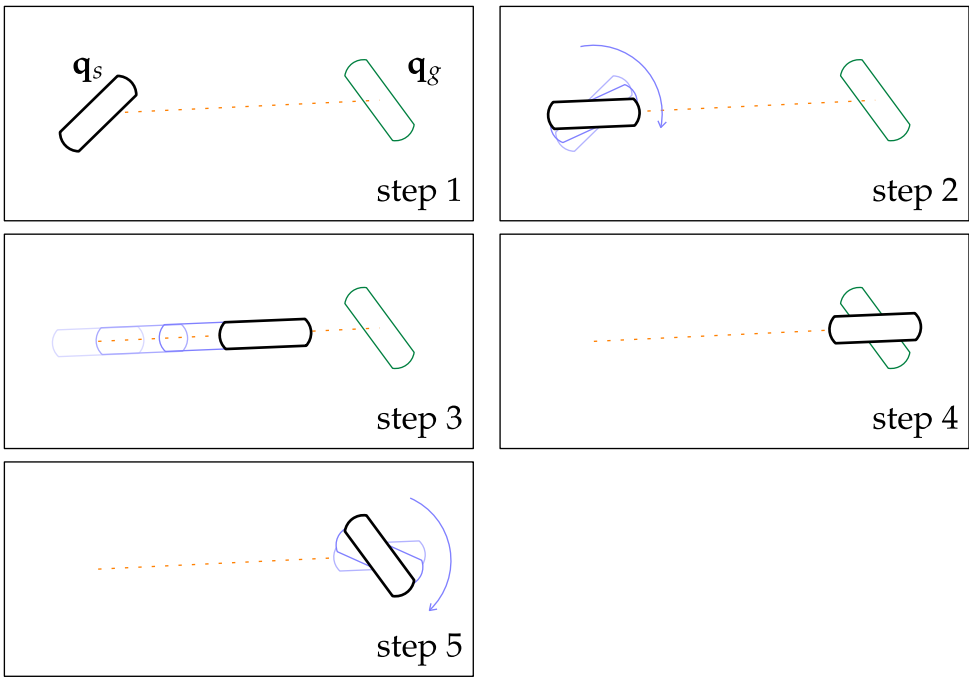
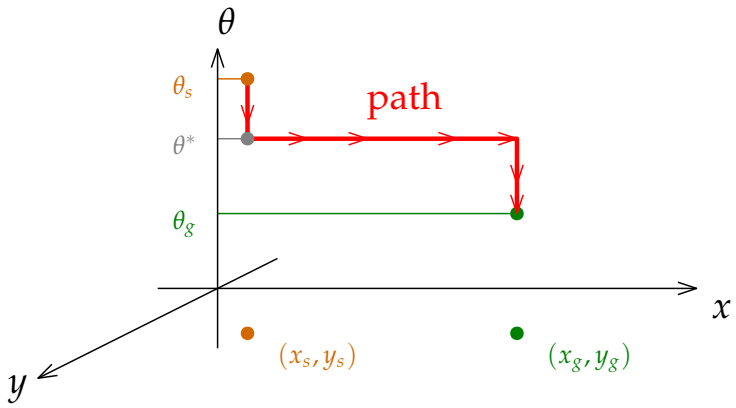


Figure 1.9: The universal maneuver in five steps.

There are no global mobility limitations. We can also observe this trajectory in the configuration space:



Note how for simplicity we have drawn the configuration space as Euclidean. Of course it is not Euclidean because $\theta \in S^1$.

1.4 Kinematic Models

We have seen how the kinematic constraints are defined in Pfaffian form as $A(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0}$, with A being a $k \times n$ matrix. We are interested in finding a basis for the null space of $A(\mathbf{q})$:

$$\mathcal{N}(A(\mathbf{q})) = \text{span}\{\mathbf{g}_1(\mathbf{q}), \mathbf{g}_2(\mathbf{q}), \ldots, \mathbf{g}_m(\mathbf{q})\} \tag{1.37}$$

$\mathcal{N}(A(\mathbf{q}))$ has dimension $m := n - k$. Each vector $\mathbf{g}_i(\mathbf{q})$ of the basis is a vector field on $\mathcal{C}$ with values in $\mathbb{R}^n$, any admissible velocity is a linear combination of the vectors in the base:

$$\dot{\mathbf{q}} = \sum_{i=1}^{m} u_i \mathbf{g}_i(\mathbf{q}) \tag{1.38}$$

we can write any admissible velocity in compact form by using an m dimensional vector $\mathbf{u}$:

$$\dot{\mathbf{q}} = \underbrace{\begin{pmatrix} | & | & & | \\ \mathbf{g}_1(\mathbf{q}) & \mathbf{g}_2(\mathbf{q}) & \cdots & \mathbf{g}_m(\mathbf{q}) \\ | & | & & | \end{pmatrix}}_{G(\mathbf{q})} \underbrace{\begin{pmatrix} u_1 \\ \vdots \\ u_m \end{pmatrix}}_{\mathbf{u}} \tag{1.39}$$

the **kinematic model** is:

$$\dot{\mathbf{q}} = G(\mathbf{q})\mathbf{u} \tag{1.40}$$

with $\mathbf{u}$ being a velocity-level quantity. We can see the model as a dynamical system, with $\mathbf{q}$ being the n-dimensional state vector and $\mathbf{u}$ the m-dimensional input vector, of course it is a nonlinear system due to $G(\mathbf{q})$ that depends (in general) nonlinearly on $\mathbf{q}$. This system satisfies the **driftless property**:

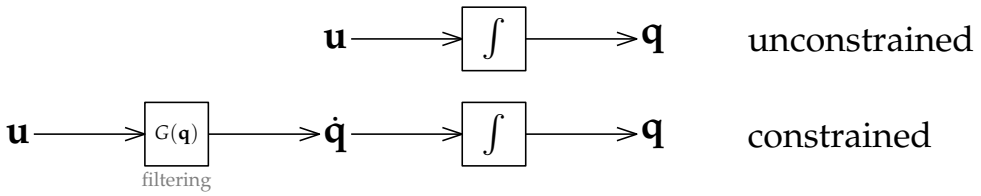
$$\mathbf{u} = \mathbf{0} \implies \dot{\mathbf{q}} = \mathbf{0} \tag{1.41}$$

it does not have a free evolution. The kinematic model is not unique, since we can select a different choice for the basis vectors $\mathbf{g}_1, \ldots, \mathbf{g}_m$, all the bases are equivalent for the description of the system, but the interpretation of the vector $\mathbf{u}$ depends on the choice of the basis. We can interpret $\mathbf{u}$ as the "virtual" velocities in the kinematically constrained configuration space.

Now we want to observe the differences between a kinematically constrained robot and an unconstrained robot. Consider an open-chain manipulator with revolute joints, with $\mathbf{q}$ being the joint variables (DH coordinates). This choice of coordinates makes the manipulator free from any constraint, the configuration can "fly" in the configuration space, any joint velocity $\dot{\mathbf{q}}$ is always admissible, hence its kinematic model is

$$\dot{\mathbf{q}} = \mathbf{u}. \tag{1.42}$$

This is a simple integrator, we can impose any joint velocity. For a robot with non-holonomic constraint this does not hold. The kinematic model is not a simple integrator since it has a *filtering* component given by the matrix $G(\mathbf{q})$, the vector $\mathbf{u}$ does not represent directly the joint velocity, but is called **pseudo-velocity**.



Let's discuss about controllability of the constrained model. The controllability of a system can be proved formally through nonlinear control theory, or constructively, by the definition of a universal maneuver (as we did in the previous section).

Definition 1.4.1 : Controllability

The kinematic model of a manipulator is controllable if any configuration can be reached from any initial configuration in finite time.

A holonomic constraint defines a geometric constraint that prevents the robot from accessing a specific region in the configuration space, hence a robot with geometric constraints is never controllable. By this reasoning, there is a 1-to-1 correspondence between controllability and the presence of holonomic constraints.

Observation 1.4.1

The kinematic model of a robot is controllable **if and only if** the set of kinematic constraints is non-holonomic.

1.4.1 The Unicycle Model

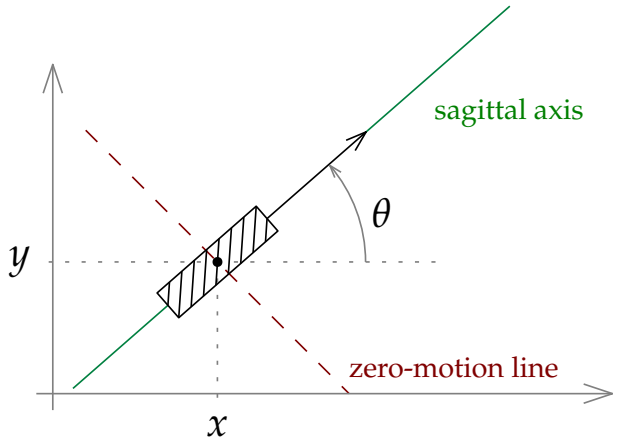
Let's define the kinematic model of a unicycle, that is equivalent to the rolling disk seen in Section 1.3.3. It has one kinematic constraint, the pure rolling condition imposes that:

$$\sin\theta\dot{x} - \cos\theta\dot{y} = 0. \tag{1.43}$$

It says that the velocity of the contact point must be aligned with the *sagittal axis* defined by θ, i.e. it can't have components on the zero-motion line. In matrix form the constraint is:

$$\underbrace{\begin{pmatrix}\sin\theta & -\cos\theta & 0\end{pmatrix}}_{A(\mathbf{q})}\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = 0. \tag{1.44}$$

The dimension of $\mathcal{N}(A)$ is 2, hence we will have a 2-dimensional vector of the pseudo-velocity.



A basis for the null space of A is the following:

$$\mathbf{g}_1(\mathbf{q}) = \begin{pmatrix}\cos\theta\\ \sin\theta\\ 0\end{pmatrix}, \quad \mathbf{g}_2(\mathbf{q}) = \begin{pmatrix}0\\ 0\\ 1\end{pmatrix} \tag{1.45}$$

the kinematic model is

$$\dot{\mathbf{q}} = G(\mathbf{q})\mathbf{u} = \begin{pmatrix}\cos\theta & 0\\ \sin\theta & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}u_1\\ u_2\end{pmatrix} \implies \begin{cases}\dot{x} = \cos\theta u_1\\ \dot{y} = \sin\theta u_1\\ \dot{\theta} = u_2\end{cases}. \tag{1.46}$$

u_1 is the scalar velocity along the sagittal axis

$$u_1 = \pm\sqrt{\dot{x}^2 + \dot{y}^2} \tag{1.47}$$

while u_2 is exactly $\dot{\theta}$. We rename these quantities:

$$v = u_1 \tag{1.48}$$
$$\omega = u_2 \tag{1.49}$$

v is related to the rolling velocity ω_{roll} and the radius of the wheel:

$$v = \omega_{\text{roll}} \cdot r. \tag{1.50}$$

We can rewrite the kinematic model by splitting the Cartesian part from the angular one:

$$\dot{\mathbf{q}} = \begin{pmatrix}\cos\theta\\ \sin\theta\\ 0\end{pmatrix} v + \begin{pmatrix}0\\ 0\\ 1\end{pmatrix}\omega. \tag{1.51}$$

Is this model controllable? Yes, we already proved for the rolling disk that a universal maneuver exists.

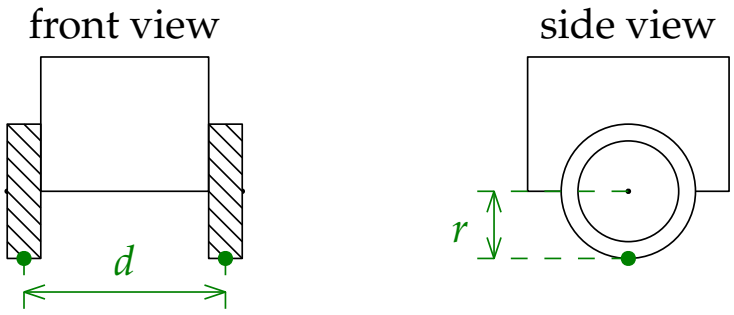
What are the real world robots that are kinematically and mechanically equivalent to the unicycle? First of all the differential drive, since it can rotate along its vertical axis, and move in the Cartesian plane only towards its sagittal axis (it can't slide laterally). It has two actuated wheels, of radius r and rolling velocities ω_R, ω_L. Its Cartesian velocity v is given by the average angular velocity of the two wheels:

$$v = r\frac{\omega_R + \omega_L}{2} \tag{1.52}$$

while its angular velocity is given by the difference:

$$\omega = (\omega_R - \omega_L)\frac{r}{d} \tag{1.53}$$

where d is the distance between the contact points of the wheels.



The other robot that is equivalent to the unicycle is the synchro-drive, it has two motors equipped with a pulley and belt mechanism, one controls the orientation of all the wheels, the other the rolling speed, hence these motors directly controls v and ω, where v is relative to the velocity of the geometric center of the three contact points.

1.4.2 The Bicycle Model

This kinematic model has two wheels, one fixed at the back (called rear wheel) and one steerable in the front.

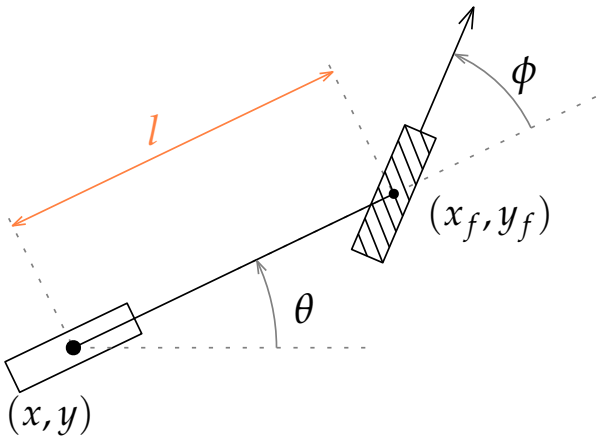


Figure 1.10: The bicycle model.

As described in Figure 1.10, the configuration space of the bicycle is $\mathcal{C} = \mathbb{R}^2 \times SO(2) \times SO(2)$, the dimension is $n = 4$. l is the distance between the two contact points of the two wheels, the first identified by (x, y), the second by (x_f, y_f), clearly:

$$x_f = x + l\cos\theta \tag{1.54}$$

$$y_f = y + l\sin\theta \tag{1.55}$$

the configuration vector is

$$\mathbf{q} = \begin{pmatrix} x & y & \theta & \phi \end{pmatrix}^\top. \tag{1.56}$$

Since there are two wheels, we will have two constraints, that imposes the pure rolling condition both wheels. For the rear wheel and the front wheel, respectively::

$$\sin\theta\dot{x} - \cos\theta\dot{y} = 0 \tag{1.57}$$
$$\sin(\theta+\phi)\dot{x}_f - \cos(\theta+\phi)\dot{y}_f = 0 \tag{1.58}$$

in vector form:

$$\underbrace{\begin{pmatrix}\sin\theta & -\cos\theta\end{pmatrix}}_{\mathbf{a}_1^\top}\begin{pmatrix}\dot{x}\\ \dot{y}\end{pmatrix} = 0 \tag{1.59}$$

$$\underbrace{\begin{pmatrix}\sin(\theta+\phi) & -\cos(\theta+\phi)\end{pmatrix}}_{\mathbf{a}_2^\top}\begin{pmatrix}\dot{x}_f\\ \dot{y}_f\end{pmatrix} = 0 \tag{1.60}$$

$\mathbf{a}_1$ and $\mathbf{a}_2$ identify the direction of the (forbidden) slide direction for both wheels. The two lines meet at the point known as **instantaneous center of rotation (ICR)**, as shown in Figure 1.11.

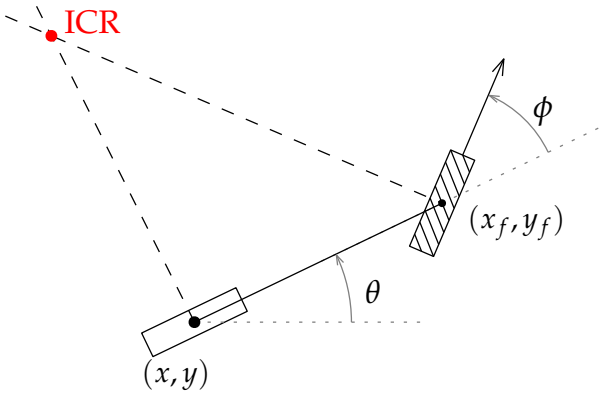


Figure 1.11: The instantaneous center of rotation.

The ICR represents the geometric point around which all the parts of the robot are rotating at a given instant. Now we want to make explicit $(\dot{x}_f, \dot{y}_f)$ in order to write the constraints only as a function of $\mathbf{q}$.

$$\begin{cases} x_f = x + l\cos\theta \\ y_f = y + l\sin\theta \end{cases} \implies \tag{1.61}$$

$$\begin{cases} \dot{x}_f = \dot{x} - \dot{\theta}l\sin\theta \\ \dot{y}_f = \dot{y} + \dot{\theta}l\cos\theta \end{cases} \tag{1.62}$$

we rewrite the constraint in Equation (1.58):

$$\sin(\theta+\phi)\dot{x}_f - \cos(\theta+\phi)\dot{y}_f = 0 \implies \tag{1.63}$$
$$\sin(\theta+\phi)(\dot{x} - \dot{\theta}l\sin\theta) - \cos(\theta+\phi)(\dot{y} + \dot{\theta}l\cos\theta) = 0 \implies \tag{1.64}$$
$$\sin(\theta+\phi)\dot{x} - \cos(\theta+\phi)\dot{y} - l\dot{\theta}\cos\phi = 0. \tag{1.65}$$

The constraints in matrix form are:

$$\underbrace{\begin{pmatrix}\sin\theta & -\cos\theta & 0 & 0\\ \sin(\theta+\phi) & -\cos(\theta+\phi) & -l\cos\phi & 0\end{pmatrix}}_{A(\mathbf{q})}\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\\ \dot{\phi}\end{pmatrix} = \mathbf{0} \tag{1.66}$$

with $\dim\mathcal{N}(A) = 2$, we should find two vectors for the basis of $\mathcal{N}(A)$. Since the last column of A is zero, one of the two vectors is:

$$\mathbf{g}_2(\mathbf{q}) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}. \tag{1.67}$$

We want to find $\mathbf{g}_1(\mathbf{q})$, a vector similar to the one used for the unicycle is

$$\mathbf{g}_1(\mathbf{q}) = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \alpha \\ 0 \end{pmatrix} \tag{1.68}$$

with α a term that has yet to be chosen. We see that:

$$\begin{pmatrix} \sin\theta & -\cos\theta & 0 & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & -l\cos\phi & 0 \end{pmatrix} \begin{pmatrix} \cos\theta \\ \sin\theta \\ \alpha \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ \sin(\theta+\phi)\cos\theta - \cos(\theta+\phi)\sin\theta - l\cos\phi\alpha \end{pmatrix} \tag{1.69}$$

now we choose α to make the second component zero:

$$\sin(\theta+\phi)\cos\theta - \cos(\theta+\phi)\sin\theta - l\cos\phi\alpha = 0 \implies \tag{1.70}$$

$$\sin\phi - l\cos\phi\alpha = 0 \implies \tag{1.71}$$

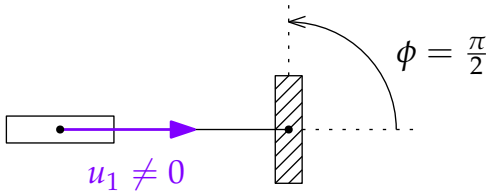
$$\alpha = \frac{1}{l}\tan\phi \tag{1.72}$$

we have found our basis, now we can write out the kinematic model:

$$\dot{\mathbf{q}} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{pmatrix} = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \frac{1}{l}\tan\phi \\ 0 \end{pmatrix} u_1 + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u_2 \tag{1.73}$$

$u_1 = \pm\sqrt{\dot{x}^2 + \dot{y}^2}$ is the magnitude of the velocity of the rear wheel contact point. If the robot is driven by the front wheel this model isn't appropriate, since u_1 is the velocity of the rear wheel.

This model has a singularity, since $\tan\phi$ isn't defined at $\phi = \pm\frac{\pi}{2}$, hence $\mathbf{g}_1$ can't be evaluated, this is due to the fact that the model assumes only pure rolling for the wheels, and if $\phi = \frac{\pi}{2}$ any non zero u_1 will make the front wheel slide.



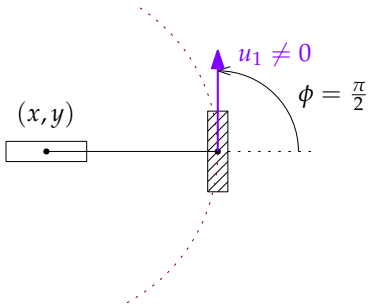
This problem is easily avoided by considering a limits on the joint values. Now let's derive a model where u_1 is the velocity of the front wheel, to do so, we have to modify the first vector of the basis:

$$\hat{\mathbf{g}}_1 = \mathbf{g}_1\cos\phi \tag{1.74}$$

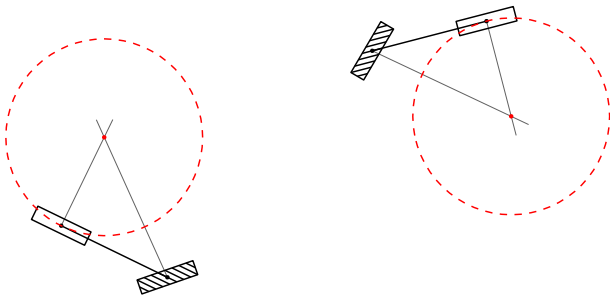
then

$$\dot{\mathbf{q}} = \underbrace{\begin{pmatrix} \cos\theta\cos\phi \\ \sin\theta\cos\phi \\ \frac{1}{l}\sin\phi \\ 0 \end{pmatrix}}_{\hat{\mathbf{g}}_1} u_1 + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u_2 \tag{1.75}$$

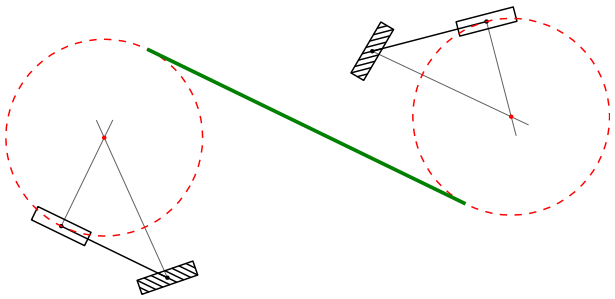
now u_1 is $\pm\sqrt{\dot{x}_f^2 + \dot{y}_f^2}$. Now the singularity on ϕ disappears, here a non zero u_1 when $\phi = \frac{\pi}{2}$ represents a motion around (x, y).



The bicycle constraints are non-holonomic and the model is controllable, we prove this fact by showing (in a non formal way) a universal maneuver. Consider two generic configurations $\mathbf{q}_s, \mathbf{q}_g$, we can draw the circles centered in the ICR that represent the Cartesian trajectory that the robot will perform by actuating just the rear wheel.



The robot in $\mathbf{q}_s$ (left) can start tracking the circular trajectory until it is on the *common tangent* between the two circles.



There, it will steer by making $\phi = 0$ and it will track the common tangent in order to reach the circular path of $\mathbf{q}_g$. There, it will steer again and reach the goal configuration.

Which robots are mechanically equivalent to a bicycle? One example is the tricycle. A tricycle with two rear wheels and one steerable front wheel can be modeled as a bicycle by defining the midpoint between the two rear wheels as (x, y). A car-like robot can also be represented by the bicycle model, but it presents a kinematic challenge: because it has two steerable front wheels, their respective zero-motion lines might intersect the zero-motion line of the rear axle at different points. When this happens, a single Instantaneous Center of Rotation (ICR) cannot be identified. This situation is illustrated in Figure 1.12.

To overcome this problem, we use a mechanism called **Ackermann steering**, which, during a circular path, automatically orients the front wheels so that their zero-motion lines intersect at a single ICR.

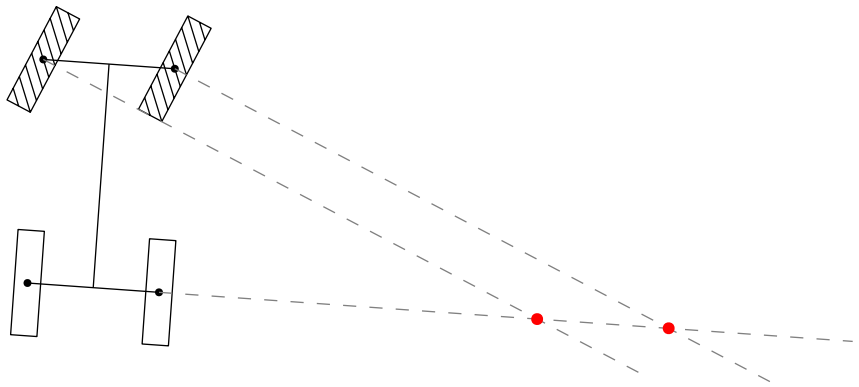


Figure 1.12: The intersections identifies two ICR.

1.5 Nonlinear Controllability

Let's discuss about the controllability of a nonlinear *driftless* system:

$$\dot{\mathbf{x}} = \sum_{i=1}^{m} \mathbf{g}_i(\mathbf{x}) u_i \tag{1.76}$$

with $\mathbf{g}_i : \mathbb{R}^n \to \mathbb{R}^n$ being a vector field and $u_1, \dots, u_m$ the inputs. In this general discussion we use the symbol $\mathbf{x}$ to denote the state of our system, although we usually use $\mathbf{q}$ (in the robotics domain).

Definition 1.5.1 : Controllability of a Nonlinear System

The system in Equation (1.76) is **controllable** if, for any pair of states $(\mathbf{x}_i, \mathbf{x}_f)$, with $\mathbf{x}(0) = \mathbf{x}_i$, there exists a profile $\tilde{\mathbf{u}}(t)$ for the inputs in $t \in [0, T]$ such that the system will reach at the end the state $\mathbf{x}(T) = \mathbf{x}_f$.

In linear systems, the controllability is guaranteed when the controllability matrix of the system have full rank, we will see how also for the nonlinear case, the controllability test will result in the derivation of the rank of a matrix.

Definition 1.5.2 : Lie Brackets

Let $\mathbf{g}_1, \mathbf{g}_2$ be two vector fields on $\mathbb{R}^n$. The **Lie Bracket** of these vector fields is the new vector field $[\mathbf{g}_1, \mathbf{g}_2]$ defined as follows:

$$[\mathbf{g}_1, \mathbf{g}_2](\mathbf{x}) = \frac{\partial \mathbf{g}_2}{\partial \mathbf{x}} \mathbf{g}_1(\mathbf{x}) - \frac{\partial \mathbf{g}_1}{\partial \mathbf{x}} \mathbf{g}_2(\mathbf{x}). \tag{1.77}$$

Let's see an example in $\mathbb{R}^3$:

$$\mathbf{g}_1 = \begin{pmatrix} \cos x_3 \\ \sin x_3 \\ 0 \end{pmatrix}, \quad \mathbf{g}_2 = \begin{pmatrix} 0 \\ 0 \\ x_1 \end{pmatrix} \tag{1.78}$$

the Jacobians are:

$$\frac{\partial \mathbf{g}_1}{\partial \mathbf{x}} = \begin{pmatrix} 0 & 0 & -\sin x_3 \\ 0 & 0 & \cos x_3 \\ 0 & 0 & 0 \end{pmatrix}, \quad \frac{\partial \mathbf{g}_2}{\partial \mathbf{x}} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \tag{1.79}$$

hence:

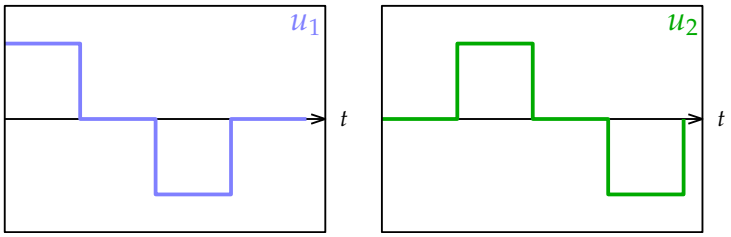
$$[\mathbf{g}_1, \mathbf{g}_2](\mathbf{x}) = \begin{pmatrix} x_1 \sin x_3 \\ -x_1 \cos x_3 \\ \cos x_3 \end{pmatrix} \tag{1.80}$$

notice how in this case $[\mathbf{g}_1, \mathbf{g}_2]$ is linearly independent from $\mathbf{g}_1, \mathbf{g}_2$. Now we want to give an *interpretation* of the Lie bracket. Consider the following 2-inputs system:

$$\dot{\mathbf{x}} = \mathbf{g}_1(\mathbf{x})u_1 + \mathbf{g}_2(\mathbf{x})u_2 \tag{1.81}$$

the interpretation of the Lie brackets can be defined by considering the following time profile for u_1, u_2 in $t \in [0, 4\varepsilon]$:

$$(u_1, u_2) = (1, 0) \quad t \in [0, \varepsilon] \tag{1.82}$$
$$(u_1, u_2) = (0, 1) \quad t \in [\varepsilon, 2\varepsilon] \tag{1.83}$$
$$(u_1, u_2) = (-1, 0) \quad t \in [2\varepsilon, 3\varepsilon] \tag{1.84}$$
$$(u_1, u_2) = (0, -1) \quad t \in [3\varepsilon, 4\varepsilon]. \tag{1.85}$$



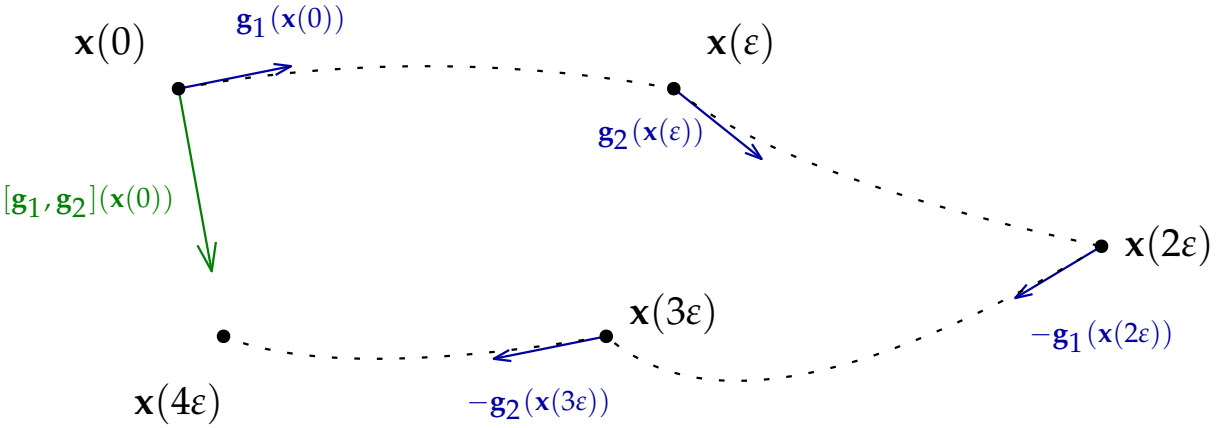
After applying this input profile the system will be in the state:

$$\mathbf{x}(4\varepsilon) = \mathbf{x}(0) + \underbrace{\varepsilon^2[\mathbf{g}_1, \mathbf{g}_2](\mathbf{x}(0)) + O(\varepsilon^3)}_{\text{displacement}}. \tag{1.86}$$

This result affirms that the displacement will be in the direction of the Lie bracket, with an error given by the term $O(\varepsilon^3)$: if $\varepsilon \to 0$, the terms in $O(\varepsilon^3)$ will vanish faster, and the displacement will be defined mainly by $\varepsilon^2[\mathbf{g}_1, \mathbf{g}_2](\mathbf{x}(0))$.

Observation 1.5.1

The Lie bracket gives the direction of the displacement of that particular input sequence.



Definition 1.5.3 : Distribution

Let $\mathcal{C}$ be a smooth manifold. A *Distribution* Δ is a map that assign at each point $\mathbf{x} \in \mathcal{C}$ a subspace of the tangent space $T_{\mathbf{x}}\mathcal{C}$:

$$\Delta : \mathbf{x} \mapsto \Delta(\mathbf{x}) \subset T_{\mathbf{x}}\mathcal{C}. \tag{1.87}$$

The tangent space in each point of a n-manifold has dimension n, hence the dimension of the subspace $\Delta(\mathbf{x})$ can be at most n (depending on the point on where we are evaluating Δ). For a system in the form (1.76), the *input distribution* is defined as follows:

$$\Delta(\mathbf{x}) = \text{span}\{\mathbf{g}_1(\mathbf{x}), \mathbf{g}_2(\mathbf{x}), \dots, \mathbf{g}_m(\mathbf{x})\}. \tag{1.88}$$

The controllability problem for nonlinear systems is strictly correlated to a particular distribution, called the **accessibility distribution** $\Delta_{\mathcal{A}}$, defined as follows:

$$\Delta_{\mathcal{A}}(\mathbf{x}) = \text{span}\{\mathbf{g}_1(\mathbf{x}), \dots, \mathbf{g}_m(\mathbf{x}), \dots, [\mathbf{g}_i, \mathbf{g}_j](\mathbf{x}), \dots, [\mathbf{g}_i, [\mathbf{g}_j, \mathbf{g}_k]](\mathbf{x}), \dots\}. \tag{1.89}$$

Let's break it down:

- With $[\mathbf{g}_i, \mathbf{g}_j]$ we denote all Lie brackets given by all the possible pairs of vector fields for $1 \leq i, j \leq m$. These are also called first order Lie brackets.
- Each pair $\mathbf{g}_i, \mathbf{g}_j$ (with $i \neq j$) appears only one time, we don't have to add also $[\mathbf{g}_j, \mathbf{g}_i]$ since it is equal to $-[\mathbf{g}_i, \mathbf{g}_j]$ (they are linearly dependent, the span needs only one of them).
- $[\mathbf{g}_i, [\mathbf{g}_j, \mathbf{g}_k]]$ for $1 \leq i, j, k \leq m$ are the second order Lie brackets.
- This nested definition can continue, for example the third order Lie brackets are in the form $[\mathbf{g}_i, [\mathbf{g}_j, [\mathbf{g}_k, \mathbf{g}_h]]]$.
- The definition of $\Delta_{\mathcal{A}}$ consider all the possible combination of Lie brackets, for each possible order, up to infinity.

Of course $\Delta_{\mathcal{A}}(\mathbf{x})$ can have different dimension depending on $\mathbf{x}$, for a nonlinear system, to assert controllability, is *mandatory* that $\Delta_{\mathcal{A}}(\mathbf{x})$ have dimension n on every single $\mathbf{x} \in \mathcal{C}$.

Definition 1.5.4 : Dimension of a Distribution

The distribution $\Delta_{\mathcal{A}}$ have dimension k if $\dim \Delta_{\mathcal{A}}(\mathbf{x}) = k$ for every $\mathbf{x} \in \mathcal{C}$. In that case we write $\dim \Delta_{\mathcal{A}} = k$.

The nonlinear controllability problem is fully described by the following result.

Theorem 1.5.1 : Accessibility Criterion

A nonlinear system of the form (1.76) with an n-dimensional state vector is controllable *if and only if* its accessibility distribution have dimension n:

$$\dim \Delta_{\mathcal{A}} = n. \tag{1.90}$$

So in practice, we have to build the matrix M which the columns are the vector fields $\mathbf{g}_i$ and all their Lie brackets, and check if the rank is full.

$$M(\mathbf{x}) = \begin{pmatrix} | & & | & & | & & | \\ \mathbf{g}_1(\mathbf{x}) & \dots & \mathbf{g}_m(\mathbf{x}) & \dots & [\mathbf{g}_i, \mathbf{g}_j](\mathbf{x}) & \dots & [\mathbf{g}_i, [\mathbf{g}_j, \mathbf{g}_k]](\mathbf{x}) \dots \\ | & & | & & | & & | \end{pmatrix} \tag{1.91}$$

It's impossible to write out all the columns, we can check each iteratively for the vector fields $\mathbf{g}_1, \dots, \mathbf{g}_m$, then for the first order Lie brackets, then for the higher order, as soon as the rank of the matrix reaches size n, the criterion is satisfied ($\dim \Delta_{\mathcal{A}} = n$) and the system is controllable. All remaining infinite columns can be ignored. For systems described by analytic functions, the chain of Lie brackets stabilizes after a finite number of steps. If at a certain point the new Lie brackets no longer increase the rank and it remains lower than n, the process stops and the system is not controllable.

Proposition 1.5.1

If the system is analytical, the accessibility distribution stabilizes. The chain of Lie brackets cannot generate distributions of infinitely increasing size in a space of dimension n. Consequently, in the worst case, it is not necessary to go beyond the Lie brackets of order $n-1$ (or in any case limited according to the size of the state).

In computational practice (for example using symbolic calculation software such as MATLAB or Python), the brackets are calculated in ascending order of degree and those that do not increase the rank are immediately discarded, blocking the algorithm as soon as the rank is equal to n.

1.5.1 The Unicycle Case

The unicycle model is:

$$\dot{\mathbf{q}} = \underbrace{\begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix}}_{\mathbf{g}_1} v + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}}_{\mathbf{g}_2} \omega \tag{1.92}$$

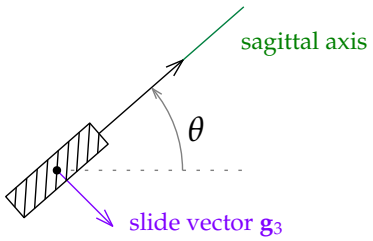
we compute the first order Lie bracket:

$$\mathbf{g}_3(\mathbf{q}) := [\mathbf{g}_1, \mathbf{g}_2](\mathbf{q}) = -\begin{pmatrix} 0 & 0 & -\sin\theta \\ 0 & 0 & \cos\theta \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \sin\theta \\ -\cos\theta \\ 0 \end{pmatrix} \tag{1.93}$$

we denote $\mathbf{g}_3$ the Lie bracket, notice how is the unit vector orthogonal to the sagittal axis, called the **slide vector field**. Let's check the matrix given by $\mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3$:

$$\operatorname{rank}\begin{pmatrix} \cos\theta & 0 & \sin\theta \\ \sin\theta & 0 & -\cos\theta \\ 0 & 1 & 0 \end{pmatrix} = \operatorname{rank}\begin{pmatrix} \cos\theta & \sin\theta & 0 \\ \sin\theta & -\cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} = 3 \tag{1.94}$$

The rank is full, we don't have to check the higher order Lie brackets, hence the system is controllable.



In this case the Lie bracket defines a velocity that is immediately unrealizable, but specifies a direction achievable through a sequence of motions.

1.5.2 The Bicycle Case

The model of the bicycle is:

$$\dot{\mathbf{q}} = \underbrace{\begin{pmatrix} \cos\theta \\ \sin\theta \\ \frac{1}{l}\tan\phi \\ 0 \end{pmatrix}}_{\mathbf{g}_1} u_1 + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}}_{\mathbf{g}_2} u_2 \tag{1.95}$$

the first Lie bracket (we don't show the computation) is:

$$\mathbf{g}_3(\mathbf{q}) := [\mathbf{g}_1, \mathbf{g}_2](\mathbf{q}) = \begin{pmatrix} 0 \\ 0 \\ -\frac{1}{l\cos^2\phi} \\ 0 \end{pmatrix} \tag{1.96}$$

this vector is called **wriggle** and is clearly linearly independent from the others, hence the rank is at least 3. Let's compute the second order Lie bracket:

$$\mathbf{g}_4 := [\mathbf{g}_1, \mathbf{g}_3](\mathbf{q}) = \begin{pmatrix} -\frac{\sin\theta}{l\cos^2\phi} \\ \frac{\cos\theta}{l\cos^2\phi} \\ 0 \\ 0 \end{pmatrix} \tag{1.97}$$

$\mathbf{g}_4$ is parallel to the slide vector field. It can be verified that:

$$\text{rank} \begin{pmatrix} \cos\theta & 0 & 0 & -\frac{\sin\theta}{l\cos^2\phi} \\ \sin\theta & 0 & 0 & \frac{\cos\theta}{l\cos^2\phi} \\ \frac{1}{l}\tan\phi & 0 & -\frac{1}{l\cos^2\phi} & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} = 4 \tag{1.98}$$

hence the model is controllable.

1.5.3 Chained-Form

Definition 1.5.5 : Chained-Form Systems

Consider a nonlinear system with an n-dimensional state vector $\mathbf{z} \in \mathbb{R}^n$ and 2 inputs $\mathbf{w} \in \mathbb{R}^2$. The system is in chained-form if has the following structure:

$$\dot{z}_1 = w_1 \tag{1.99}$$
$$\dot{z}_2 = w_2 \tag{1.100}$$
$$\dot{z}_3 = z_2 w_1 \tag{1.101}$$
$$\dot{z}_4 = z_3 w_1 \tag{1.102}$$
$$\vdots \tag{1.103}$$
$$\dot{z}_n = z_{n-1} w_1. \tag{1.104}$$

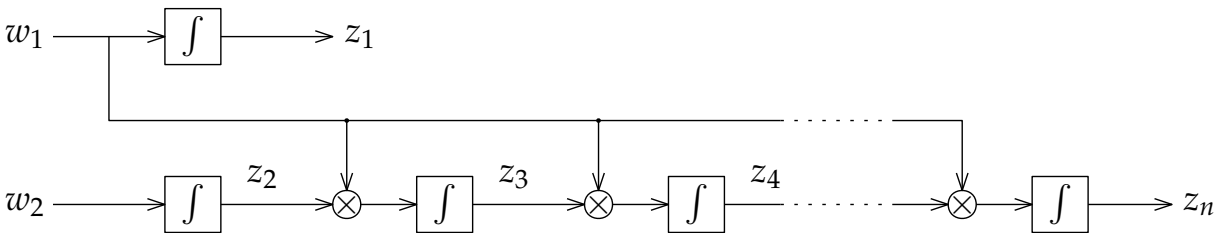


Figure 1.13: Block diagram of a chained-form system.

We can observe how, if the inputs $\mathbf{w}$ are constant, the system becomes linear, hence in digital control, where the inputs are piece-wise constants, the system is piece-wise linear.

Proposition 1.5.2

Any chained-form system is controllable.

All the controllable driftless systems (that are the systems of our interest for the kinematic model of the mobile robots) can be rewritten in chained-form under a coordinate and inputs transformation:

$$\mathbf{z} = \alpha(\mathbf{q}) \tag{1.105}$$
$$\mathbf{w} = \beta(\mathbf{q})\mathbf{u}. \tag{1.106}$$

Now we will show how to transform the unicycle model in chained-form. The original model is:

$$\dot{x} = v\cos\theta \tag{1.107}$$
$$\dot{y} = v\sin\theta \tag{1.108}$$
$$\dot{\theta} = \omega \tag{1.109}$$

we define the following transformation for the coordinates:

$$\mathbf{z} = \alpha(\mathbf{q}) = \begin{cases} z_1 = \theta \\ z_2 = x\cos\theta + y\sin\theta \\ z_3 = x\sin\theta - y\cos\theta \end{cases} \tag{1.110}$$

and the following transformation for the inputs:

$$\mathbf{w} = \beta(\mathbf{q})\begin{pmatrix} v \\ \omega \end{pmatrix} = \begin{cases} w_1 = \omega \\ w_2 = v - z_3\omega \end{cases} . \tag{1.111}$$

Now we rewrite the model and see if it is in chained-form. From Equation (1.110) we get:

$$\dot{z}_1 = \dot{\theta} = \omega = w_1 \tag{1.112}$$

then:

$$\dot{z}_2 = \underbrace{\dot{x}\cos\theta + \dot{y}\sin\theta}_{v} - x\sin\theta\dot{\theta} + y\cos\theta\dot{\theta} \tag{1.113}$$
$$= v - \underbrace{\dot{\theta}}_{\omega}\underbrace{(x\sin\theta - y\cos\theta)}_{z_3} = \underbrace{v - \omega z_3 = w_2}_{\text{input transformation}} \tag{1.114}$$

ultimately:

$$\dot{z}_3 = \dot{x}\sin\theta - \dot{y}\cos\theta - x\cos\theta\dot{\theta} + y\sin\theta\dot{\theta} \tag{1.115}$$
$$= \underbrace{(\dot{x}\sin\theta - \dot{y}\cos\theta)}_{0 \text{ due to the constraint}} + \dot{\theta}(x\cos\theta + y\sin\theta) \tag{1.116}$$
$$= \underbrace{\dot{\theta}}_{\omega = w_1}\underbrace{(x\cos\theta + y\sin\theta)}_{z_2} = z_2 w_1 \tag{1.117}$$

so the model after the transformations is:

$$\dot{z}_1 = w_1 \tag{1.118}$$
$$\dot{z}_2 = w_2 \tag{1.119}$$
$$\dot{z}_3 = z_2 w_1 \tag{1.120}$$

and it is in chained-form. z_2 and z_3 have a clear interpretation, are the coordinates of the unicycle in the reference frame with its horizontal axis rotated by θ with respect to the base frame, as shown in Figure 1.14.

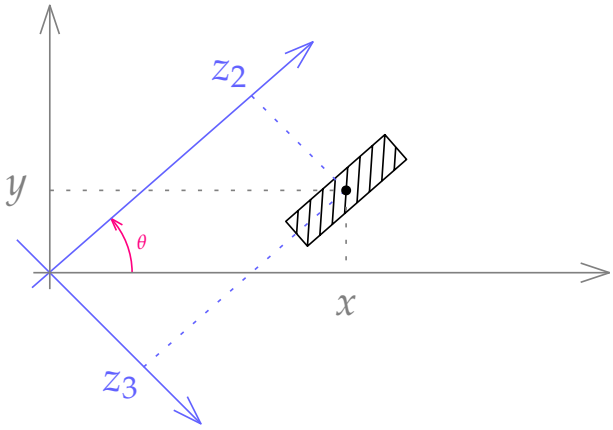


Figure 1.14: Interpretation of z_2 and z_3.

Setting aside the unicycle model: an interesting property of **trailed vehicles** emerges when considering a car-like robot: if the trailer is hitched exactly at the midpoint between the two rear wheels (denoted by the point (x, y)), the resulting kinematic model can still be converted into chained-form. This property holds recursively if the trailer itself has two rear wheels and an additional trailer is hitched at their midpoint, the overall kinematic model can still be expressed in chained-form.

1.6 Path and Trajectory Planning

It is in our interest to define a method to *plan* a motion that transfers a mobile robot from an initial configuration $\mathbf{q}_i$ to a final configuration $\mathbf{q}_f$, hence, we should define a trajectory:

$$\mathbf{q}(s), \quad s \in [s_i, s_f] \tag{1.121}$$

such that $\mathbf{q}(t_i) = \mathbf{q}_i$ and $\mathbf{q}(t_f) = \mathbf{q}_f$, without violating the kinematic constraints on $\dot{\mathbf{q}}$. In general, as we have already seen in previous robotics courses, we separate the geometric path from the timing law:

$$\mathbf{q}(s), \quad s \in [s_i, s_f] \tag{1.122}$$

$$s(t), \quad t \in [t_i, t_f]. \tag{1.123}$$

The path parameter s is usually chosen as the arc length.

- By **path planning** we mean the plan of a geometric path $\mathbf{q}(s)$ in the configuration space.
- By **trajectory planning** instead, we also include the time, i.e. we mean the plan of a time evolution of the configuration: $\mathbf{q}(t)$. This is usually done indirectly: first plan a path, then add a timing law.

We already specified that a trajectory $\mathbf{q}(s(t))$ must satisfy the constraints:

$$\dot{\mathbf{q}} = \frac{d\mathbf{q}}{dt} = \frac{d\mathbf{q}}{ds}\dot{s} = \mathbf{q}'\dot{s} \tag{1.124}$$

we denote $\mathbf{q}'$ the derivative of $\mathbf{q}$ with respect to s, this represents the *tangent vector* of the geometric path in the configuration space. If s is the arc length, then $\|\mathbf{q}'\| = 1$.

Example: Consider a robot with $\mathbf{q} = (x, y)$, we want to go from $(1, 0)$ to $(0, 1)$, one possible choice for the geometric path is:

$$\mathbf{q}(s) = \begin{cases} \cos s \\ \sin s \end{cases} \tag{1.125}$$

with $s \in \left[0, \frac{\pi}{2}\right]$. This is exactly the arc length, indeed:

$$\mathbf{q}'(s) = \begin{cases} -\sin s \\ \cos s \end{cases} \implies \|\mathbf{q}'(s)\| = \sqrt{(-\sin s)^2 + \cos^2 s} = 1. \tag{1.126}$$

Another choice of parametrization is

$$\mathbf{q}(s) = \begin{cases} \sqrt{1-s^2} \\ s \end{cases} \tag{1.127}$$

with $s \in [0,1]$, here it's not the arc length:

$$\mathbf{q}'(s) = \begin{cases} -\frac{s}{\sqrt{1-s^2}} \\ 1 \end{cases} \implies \|\mathbf{q}'(s)\| = \frac{1}{\sqrt{1-s^2}}. \tag{1.128}$$

Now we take into account the kinematic constraints, we know that:

$$A(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \tag{1.129}$$

$\dot{\mathbf{q}}$ is the time derivative of the planned trajectory $\mathbf{q}(s(t))$:

$$A(\mathbf{q})\dot{\mathbf{q}} = A(\mathbf{q})\mathbf{q}'\dot{s} = \mathbf{0} \implies A(\mathbf{q})\mathbf{q}' = \mathbf{0} \tag{1.130}$$

we have assumed $\dot{s} \neq 0$, the result is that the tangent vector $\mathbf{q}'$ must lie in the null space of A:

$$\mathbf{q}' \in \mathcal{N}(A) \tag{1.131}$$

hence $\mathbf{q}'$ can be expressed as a linear combination of the basis vectors $\mathbf{g}_1, \ldots, \mathbf{g}_m$, exactly as the kinematic model, hence we can write the so-called **geometric model**:

$$\mathbf{q}' = G(\mathbf{q})\tilde{\mathbf{u}} \tag{1.132}$$

this model describes explicitly the admissible tangent vectors. $\tilde{\mathbf{u}}$ is the vector of the **geometric inputs**, by multiplying both sides by $\dot{s}$ we can conclude that:

$$\underbrace{\mathbf{u}(t)}_{\text{pseudo velocity}} = \tilde{\mathbf{u}}(s)\dot{s}(t). \tag{1.133}$$

After we have decided a profile for the geometric inputs $\tilde{\mathbf{u}}(s)$, we have determined a geometric path in the configuration space, only after having defined also $s(t)$ this will identify a particular trajectory.

Example: Let's apply this reasoning to the unicycle model, the pure rolling constraint for the tangent vector is:

$$\begin{pmatrix} \sin\theta & -\cos\theta & 0 \end{pmatrix} \mathbf{q}' = 0 \tag{1.134}$$

this means that the tangent vector $\mathbf{q}'$ must always be aligned with the sagittal axis of the unicycle: paths such as the one in Figure 1.15 are not admissible.

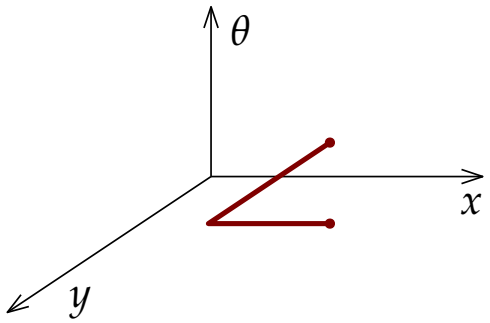


Figure 1.15: Geometric path with tangent vector non-aligned with the sagittal axis.

1.6.1 Flat Outputs

Definition 1.6.1 : Differential Flatness

Consider a general nonlinear system

$$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}. \tag{1.135}$$

The system is **differentially flat** if there exists a vector $\mathbf{y} = \Psi(\mathbf{x})$, called the **flat outputs**, such that the state and the control inputs can be expressed algebraically as a function of $\mathbf{y}$ and its time derivatives up to a finite order:

$$\mathbf{x} = \Phi_x(\mathbf{y}, \dot{\mathbf{y}}, \ddot{\mathbf{y}}, \dots, \mathbf{y}^{(r_x)}) \tag{1.136}$$

$$\mathbf{u} = \Phi_u(\mathbf{y}, \dot{\mathbf{y}}, \ddot{\mathbf{y}}, \dots, \mathbf{y}^{(r_u)}) \tag{1.137}$$

In Definition 1.6.1 we talk about time derivatives (since that is the domain of dynamical systems), but of course it also holds if all the derivatives are taken with respect to the parameter s (the case of our geometric model). The flatness property is useful in control, if we define a trajectory for $\mathbf{y}$ that is differentiable up to the $\max(r_x, r_u)$ order, we can reconstruct the state evolution $\mathbf{x}$ and the required inputs $\mathbf{u}$ through the reconstruction maps Φ_x, Φ_u.

We immediately show this property with the unicycle model.

$$x' = \tilde{v}\cos\theta \tag{1.138}$$

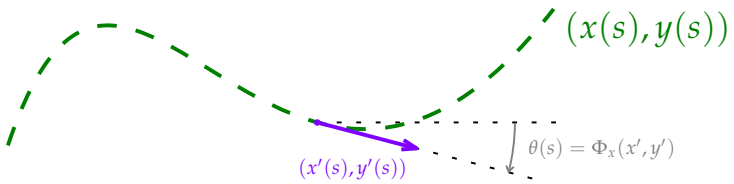
$$y' = \tilde{v}\sin\theta \tag{1.139}$$

$$\theta' = \tilde{\omega} \tag{1.140}$$

The intuition behind flatness is the following: If I know perfectly the path of the Cartesian position of the unicycle (x, y), I can infer what the path of the orientation θ is, together with the input profile that executes that path. For example, by knowing $(x(s), y(s))$, I can conclude that, at a given $s = s^*$, the orientation must be aligned with the direction of motion, i.e. the vector $(x'(s^*), y'(s^*))$.

$$\theta(s) = \Phi_x(x', y') = \text{atan2}(y'(s), x'(s)) + k\pi. \tag{1.141}$$

$k \in \{0, 1\}$ is a constant that defines the direction of motion.



Also the input $\tilde{v}$ can be reconstructed from the path of x, y, since it is exactly the norm of the Cartesian velocity:

$$\begin{cases} x' = \tilde{v}\cos\theta \\ y' = \tilde{v}\sin\theta \end{cases} \implies \tilde{v} = \pm\sqrt{(x')^2 + (y')^2} \tag{1.142}$$

the same can be done for the angular velocity (we skip the proof):

$$\tilde{\omega} = \frac{y''x' - x''y'}{(x')^2 + (y')^2}. \tag{1.143}$$

Equations (1.142),(1.143) are the reconstruction map for the input $\tilde{\mathbf{u}} = \Phi_u(x', y', x'', y'')$. These reconstructions have a problem that arises if $x' = y' = 0$: atan2 is not defined and $\tilde{\omega}$ would have a zero in the denominator. This singularity has an intuitive meaning: if the robot stops and its Cartesian velocity is zero, we cannot know whether it is steering in place or not, hence we cannot infer the evolution of $\theta(s)$ when it is not moving.

Proposition 1.6.1

A driftless dynamic system is differentially flat if and only if it can be transformed into chained-form.

Proposition 1.6.2

A general dynamic system is differentially flat if and only if it can be linearized via dynamic feedback.

The wheeled robots in our interest can be put in chained-form, hence, they admit flat outputs, this is a great simplification from a control point of view, since we can directly execute a desired trajectory through the reconstruction maps. Let's see the flat outputs for the bicycle, the model is:

$$x' = \tilde{v}\cos\theta \tag{1.144}$$
$$y' = \tilde{v}\sin\theta \tag{1.145}$$
$$\theta' = \frac{\tilde{v}}{l}\tan\phi \tag{1.146}$$
$$\phi' = \tilde{\omega} \tag{1.147}$$

x, y, θ can be reconstructed in the same way as for the unicycle case, to reconstruct ϕ we use θ':

$$\theta(s) = \text{atan2}(y', x') + k\pi \tag{1.148}$$
$$\implies \theta'(s) = \frac{x'y'' - y'x''}{(x')^2 + (y')^2} \tag{1.149}$$

hence

$$\theta' = \frac{\tilde{v}}{l}\tan\phi \tag{1.150}$$
$$\implies \phi(s) = \arctan\left(\frac{l\theta'}{\tilde{v}}\right) \tag{1.151}$$
$$= \arctan\left(\frac{l\frac{x'y''-y'x''}{(x')^2+(y')^2}}{\pm\sqrt{(x')^2+(y')^2}}\right) \tag{1.152}$$
$$= \arctan\left(\pm\frac{l(x'y''-y'x'')}{((x')^2+(y')^2)^{3/2}}\right). \tag{1.153}$$

Now we use Proposition 1.6.1 and show how a 2-input/3-state chained-form system admits flat outputs. The system is the following:

$$z_1' = \tilde{w}_1 \tag{1.154}$$
$$z_2' = \tilde{w}_2 \tag{1.155}$$
$$z_3' = z_2\tilde{w}_1 \tag{1.156}$$

for a chained-form system the flat outputs **are always the first and the last variables**:

$$\mathbf{y} = \begin{pmatrix} z_1 \\ z_3 \end{pmatrix} \tag{1.157}$$

the reconstruction maps are the following:

$$\Phi_x(z_1', z_3') = \frac{z_3'}{z_1'} \tag{1.158}$$
$$\Phi_u(z_1', z_1'', z_3', z_3'') = \begin{cases} \tilde{w}_1 = z_1' \\ \tilde{w}_2 = \dfrac{z_1'z_3'' - z_1''z_3'}{(z_1')^2} \end{cases}. \tag{1.159}$$

1.6.2 Path Planning for the Unicycle

Any flat-output profile is automatically mapped to an admissible path. An algorithm for path planning exploiting differential flatness is the following:

1. Compute the initial and final values $\mathbf{y}_i, \mathbf{y}_f$ for the flat outputs from $\mathbf{q}_i, \mathbf{q}_f$.
2. Define $\mathbf{y}(s)$ by interpolating $\mathbf{y}_i, \mathbf{y}_f$ with the appropriate boundary conditions for $\mathbf{q}$.
3. Use the reconstruction formulas to find $\mathbf{q}(s)$ and $\tilde{\mathbf{u}}(s)$.

The same algorithm is valid for trajectory planning by interpolating over t instead of s. Let's apply this algorithm to the unicycle. We use a parametrization with $s \in [0,1]$, from

$$\mathbf{q}_i = \begin{pmatrix} x_i \\ y_i \\ \theta_i \end{pmatrix}, \quad \mathbf{q}_f = \begin{pmatrix} x_f \\ y_f \\ \theta_f \end{pmatrix} \tag{1.160}$$

we extract

$$\mathbf{y}_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix}, \quad \mathbf{y}_f = \begin{pmatrix} x_f \\ y_f \end{pmatrix} \tag{1.161}$$

and perform an interpolation between these two points with a cubic polynomial function:

$$x(s) = s^3 x_f - (s-1)^3 x_i + \alpha_x s^2 (s-1) + \beta_x s (s-1)^2 \tag{1.162}$$

$$y(s) = s^3 y_f - (s-1)^3 y_i + \alpha_y s^2 (s-1) + \beta_y s (s-1)^2 \tag{1.163}$$

this polynomial already satisfies the boundary conditions:

$$x(0) = x_i, \quad x(1) = x_f \tag{1.164}$$

$$y(0) = y_i, \quad y(1) = y_f \tag{1.165}$$

then $\alpha_x, \beta_x, \alpha_y, \beta_y$ must be chosen in order to satisfy two additional boundary conditions that come from the kinematic constraints, (x', y') must be aligned with θ, and θ_i, θ_f are given, hence there are the following conditions:

$$\text{atan2}(y'(0), x'(0)) = \theta_i \tag{1.166}$$

$$\text{atan2}(y'(1), x'(1)) = \theta_f. \tag{1.167}$$

The initial and final Cartesian velocities must be:

$$x'(0) = k_i \cos\theta_i, \quad x'(1) = k_f \cos\theta_f \tag{1.168}$$

$$y'(0) = k_i \sin\theta_i, \quad y'(1) = k_f \sin\theta_f \tag{1.169}$$

with k_i, k_f being two arbitrary non-zero constants, for simplicity we set $k_i = k_f = k$. To impose these conditions, we can solve algebraically for $\alpha_x, \beta_x, \alpha_y, \beta_y$ and obtain:

$$\alpha_x = k\cos\theta_f - 3x_f, \quad \beta_x = k\cos\theta_i + 3x_i \tag{1.170}$$

$$\alpha_y = k\sin\theta_f - 3y_f, \quad \beta_y = k\sin\theta_i + 3y_i. \tag{1.171}$$

It will be $\tilde{v}(0) = \tilde{v}(1) = k$, hence k can be chosen to shape the trajectory, by affecting the length of the first and last "linear" segments of the path, as shown in Figure 1.16.

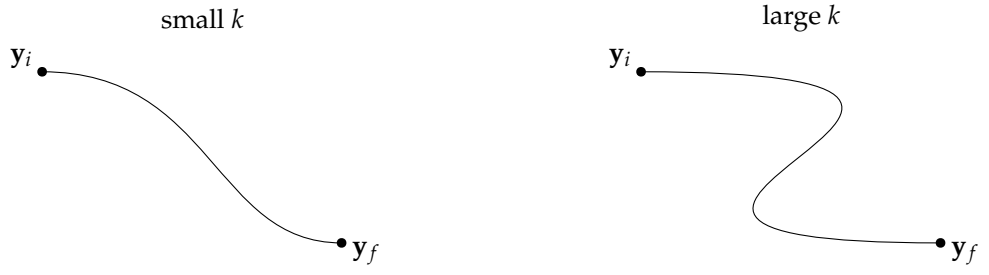


Figure 1.16: Difference between two paths with the same interpolating points but different choice of k.

1.6.3 Path Planning for a Chained-Form System

Consider a 2-input/3-state chained-form system, we apply the path planning algorithm to this case. We recall that the flat outputs are z_1, z_3, hence we have to interpolate (z_{1i}, z_{3i}) with (z_{1f}, z_{3f}). We interpolate linearly z_1 and with a cubic polynomial z_3:

$$z_1(s) = z_{1i} + s(z_{1f} - z_{1i}) \tag{1.172}$$

$$z_3(s) = s^3 z_{3f} - (s-1)^3 z_{3i} + \alpha_3 s^2(s-1) + \beta_3 s(s-1)^2 \tag{1.173}$$

that already satisfies the boundary conditions on z_1, z_3. We must impose the additional boundary conditions for z_2:

$$z_{2i} = \frac{z_3'(0)}{z_1'(0)}, \quad z_{2f} = \frac{z_3'(1)}{z_1'(1)} \tag{1.174}$$

we solve for α_3, β_3:

$$\alpha_3 = z_{2f}(z_{1f} - z_{1i}) - 3z_{3f} \tag{1.175}$$

$$\beta_3 = z_{2i}(z_{1f} - z_{1i}) + 3z_{3i} \tag{1.176}$$

After having $z_1(s), z_3(s)$ we can reconstruct $z_2(s), \tilde{w}_1(s), \tilde{w}_2(s)$ through the reconstruction maps, then $\mathbf{q}(s), \tilde{v}(s), \tilde{\omega}(s)$ are computed by inverting the chained form coordinate and input transformation.

By recalling that $z_1 = \theta$: one problem arises when the initial and final orientations are identical $z_{1i} = z_{1f}$, since we would have:

$$z_1'(s) = z_{1f} - z_{1i} = 0 \tag{1.177}$$

and then z_{2i}, z_{2f} can't be inferred since we would have a division by zero, in that case it is sufficient to set:

$$z_{1f} = z_{1i} + 2\pi \tag{1.178}$$

or add a mid-point $\mathbf{q}_m = \begin{pmatrix} x_m & y_m & \theta_m \end{pmatrix}$ in the trajectory with $\theta_m \neq \theta_i$.

1.6.4 Path Planning via Parametrized Inputs

A different approach, not directly based on flatness, consists of writing the inputs in parametrized form and then imposing that the robot goes from $\mathbf{q}_i$ to $\mathbf{q}_f$. We assume that the n-state robot model can be rewritten in chained form:

$$\begin{cases} z_1' = \tilde{w}_1 \\ z_2' = \tilde{w}_2 \\ z_3' = z_2 \tilde{w}_1 \\ \vdots \\ z_n' = z_{n-1} \tilde{w}_1 \end{cases} \tag{1.179}$$

which is linear in $\mathbf{z}$ when $\tilde{w}_1$ is constant. Let $\mathbf{z}_i, \mathbf{z}_f$ be the initial and final configurations in the chained-form system coordinates, we define Δ as the difference between the final and initial values for z_1:

$$\Delta = z_{1f} - z_{1i} \tag{1.180}$$

and we choose the following profiles for the geometric inputs:

$$\begin{cases} \tilde{w}_1 = \text{sign}(\Delta) \\ \tilde{w}_2 = \displaystyle\sum_{j=0}^{n-2} c_j s^j \end{cases} \qquad s \in [0, |\Delta|] \tag{1.181}$$

by doing so, we can integrate $z_2' = \tilde{w}_2$ analytically to get the profiles (as functions of s) of $z_2, z_3, \ldots, z_n$, and impose $z_j(|\Delta|) = z_{jf}$ for each j. This leads to a linear system of the following form:

$$D(\Delta) \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-2} \end{pmatrix} = \mathbf{d}(\mathbf{z}_i, \mathbf{z}_f, \Delta) \tag{1.182}$$

if $D(\Delta)$ is invertible we can find the values for the coefficients. For example, with a system of 2 inputs and 3 states:

$$D = \begin{pmatrix} |\Delta| & \Delta^2/2 \\ \text{sign}(\Delta)\Delta^2/2 & \Delta^3/6 \end{pmatrix}, \quad \mathbf{d} = \begin{pmatrix} z_{2f} - z_{2i} \\ z_{3f} - z_{3i} - z_{2i}\Delta \end{pmatrix} \tag{1.183}$$

1.6.5 Choice of the Timing Law

We have separated the timing law $s(t)$ from the geometric path $\mathbf{q}(s)$, now we have to assign a timing law that interpolates s_i to s_f:

$$s(t_i) = s_i, \quad s(t_f) = s_f. \tag{1.184}$$

We will select a timing law that is monotonic increasing, since it does not have any meaning for the trajectory to go backwards (in that case, this has to be defined geometrically), hence:

$$\dot{s} > 0. \tag{1.185}$$

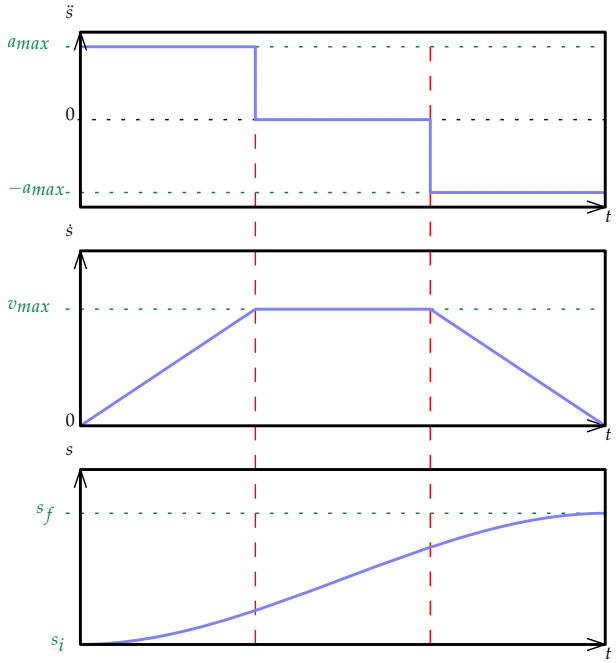
The simplest timing law is given by linear interpolation:

$$s(t) = s_i + \frac{s_f - s_i}{t_f - t_i}(t - t_i) \tag{1.186}$$

usually we use $t \in [0, T]$:

$$s(t) = s_i + (s_f - s_i)\frac{t}{T}. \tag{1.187}$$

One of the most used timing laws is the bang-coast-bang acceleration profile (**trapezoidal timing law**), since it starts and ends with zero velocity.



$$\ddot{s}(t) = \begin{cases} a_{max} & \text{if } t \in [t_i, t_i + t_m] \\ 0 & \text{if } t \in (t_i + t_m, t_f - t_m] \\ -a_{max} & \text{if } t \in (t_f - t_m, t_f] \end{cases}. \tag{1.188}$$

Now let's take into account the bounds on velocity, for example, consider a unicycle subject to the following bounds for the linear and angular velocities:

$$|v| \leq v_{max}, \quad |\omega| \leq \omega_{max} \tag{1.189}$$

assume that a path $\mathbf{q}(s)$ has already been generated, with geometric inputs $\tilde{v}(s)$ and $\tilde{\omega}(s)$, we choose a linear timing law parametrized by a positive real number k:

$$s(t) = s_i + \frac{t}{k}, \quad t \in [0, k(s_f - s_i)]. \tag{1.190}$$

We assume that $\mathbf{q}(s)$ is defined such that s is the arc length. The timing law derivative is $\dot{s} = \frac{1}{k}$, with

$$v(t) = \frac{\tilde{v}(s(t))}{k}, \quad \omega(t) = \frac{\tilde{\omega}(s(t))}{k} \tag{1.191}$$

in order to respect the bounds we have to choose k as follows:

$$k = \max \left\{ \frac{\tilde{v}_{peak}}{v_{max}}, \frac{\tilde{\omega}_{peak}}{\omega_{max}} \right\} \tag{1.192}$$

where

$$\tilde{v}_{peak} = \max_{s \in [s_i, s_f]} |\tilde{v}(s)| \tag{1.193}$$

$$\tilde{\omega}_{peak} = \max_{s \in [s_i, s_f]} |\tilde{\omega}(s)|. \tag{1.194}$$

CHAPTER

2

MOTION CONTROL

In order to follow a desired trajectory, or to regulate the robot into a desired configuration, we can't simply proceed by applying the necessary inputs computed from the flat outputs, in order to reject the disturbances and the errors we must add a feedback action. In previous robotics courses, we have considered the dynamic model of the manipulators, controlled by the torques on the joints, for wheeled mobile robotics we can consider only the kinematic model, since the dynamic model is simpler, can be canceled via feedback and the wheels are always equipped with low-level control loops that can regulate velocities in a very reliable way.

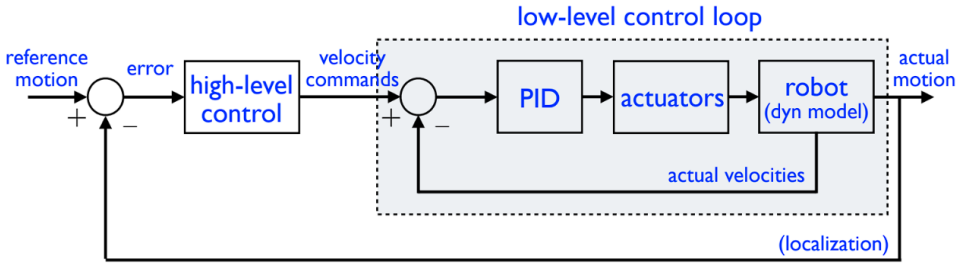


Figure 2.1: Actual control scheme.

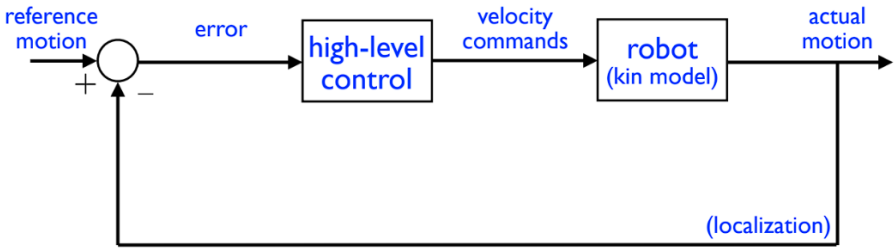


Figure 2.2: Simplified control scheme (for design).

We cover two type of control tasks, *trajectory tracking* and *regulation*, we will provide a full description of this problem for the unicycle model. In **trajectory tracking** the robot must converge to a desired Cartesian trajectory $(x_d(t), y_d(t))$, while in **posture regulation** the robot must converge to a desired con-

figuration $\mathbf{q}_d$. Trajectory tracking is more relevant in practice, since moving along planned trajectories reduces the risk of collision, however, posture regulation does not require any planning.

2.1 Trajectory Tracking

Let $(x_d(t), y_d(t))$ be an admissible desired trajectory for the unicycle, there exists control inputs v_d, ω_d such that:

$$\begin{cases} \dot{x}_d = v_d \cos\theta_d \\ \dot{y}_d = v_d \sin\theta_d \\ \dot{\theta}_d = \omega_d \end{cases} . \tag{2.1}$$

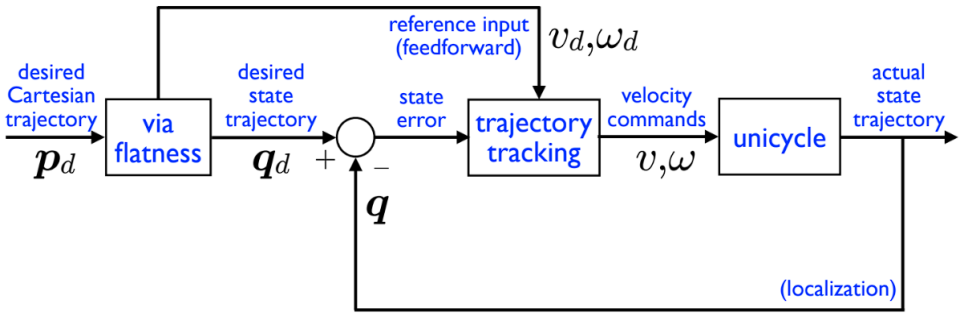
Thanks to the flatness property, from the desired trajectory $(x_d(t), y_d(t))$ we can compute the evolution of the full state and the required inputs:

$$\theta_d(t) = \text{atan2}(\dot{y}_d(t), \dot{x}_d(t)) + k\pi \quad k \in \{0, 1\} \tag{2.2}$$

$$v_d(t) = \pm\sqrt{\dot{x}_d^2(t) + \dot{y}_d^2(t)} \tag{2.3}$$

$$\omega_d(t) = \frac{\ddot{y}_d(t)\dot{x}_d(t) - \ddot{x}_d(t)\dot{y}_d(t)}{\dot{x}_d^2(t) + \dot{y}_d^2(t)}. \tag{2.4}$$

We recall that an admissible trajectory requires $(\dot{x}_d(t), \dot{y}_d(t))$ to be continuous over time, the desired state trajectory $\mathbf{q}_d(t)$ that comes from the flat outputs will be used to compute the state error, then used in the controller. The resulting block diagram will be:



The objective is to drive the state error to zero, we will express the error in a coordinate frame rotated by θ, with its x-axis parallel to the sagittal axis of the robot:

$$\mathbf{e} = \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{pmatrix} . \tag{2.5}$$

(e_1, e_2) is the Cartesian error $\mathbf{e}_p = (x_d - x,\ y_d - y)^\top$ expressed in a frame rotated by θ, shown in Figure 2.3. Differentiating $\mathbf{e}$ and using the unicycle kinematic model together with the desired one, we obtain the **error dynamics**:

$$\dot{e}_1 = v_d \cos e_3 - v + e_2\,\omega \tag{2.6}$$

$$\dot{e}_2 = v_d \sin e_3 - e_1\,\omega \tag{2.7}$$

$$\dot{e}_3 = \omega_d - \omega \tag{2.8}$$

which is **nonlinear** and **time-varying**, since it depends on the reference inputs $v_d(t), \omega_d(t)$.

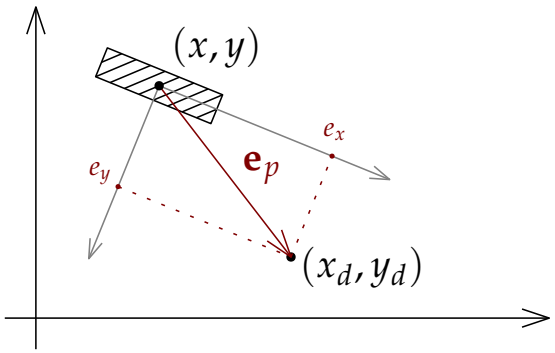


Figure 2.3: Error expressed in the rotated frame.

2.1.1 Approximate Linearization

Before applying it to the unicycle, let us recall the idea of stabilization via approximate linearization: to stabilize a nonlinear system at an equilibrium, we stabilize its approximate linearization around it. Consider a generic nonlinear system

$$\dot{\mathbf{x}} = \boldsymbol{\eta}(\mathbf{x}, \mathbf{u}) = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u} \qquad \mathbf{x} \in \mathbb{R}^n,\ \mathbf{u} \in \mathbb{R}^m \tag{2.9}$$

and assume, without loss of generality, that the origin is the desired **unforced equilibrium**, i.e. $\boldsymbol{\eta}(\mathbf{0}, \mathbf{0}) = \mathbf{0}$, or equivalently $f(\mathbf{0}) = \mathbf{0}$. The first-order Taylor expansion of $\boldsymbol{\eta}$ around $\mathbf{x} = \mathbf{0}, \mathbf{u} = \mathbf{0}$ gives

$$\boldsymbol{\eta}(\mathbf{x}, \mathbf{u}) \approx \underbrace{\boldsymbol{\eta}(\mathbf{0}, \mathbf{0})}_{=\mathbf{0}} + \left.\frac{\partial \boldsymbol{\eta}}{\partial \mathbf{x}}\right|_{\substack{\mathbf{x}=\mathbf{0} \\ \mathbf{u}=\mathbf{0}}} \mathbf{x} + \left.\frac{\partial \boldsymbol{\eta}}{\partial \mathbf{u}}\right|_{\substack{\mathbf{x}=\mathbf{0} \\ \mathbf{u}=\mathbf{0}}} \mathbf{u} \tag{2.10}$$

and the **approximate linearization** of the system at the origin is

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \tag{2.11}$$

where A and B are the two Jacobian matrices evaluated at the origin. Let now $\mathbf{u} = K\mathbf{x}$, so that

$$\dot{\mathbf{x}} = A\mathbf{x} + BK\mathbf{x} = (A + BK)\mathbf{x}. \tag{2.12}$$

If K is chosen in such a way that $A + BK$ is **Hurwitz**[1] (certainly possible if (A, B) is controllable), then the approximate linearization is asymptotically stable.

Theorem 2.1.1 : Lyapunov Indirect Method

If the approximate linearization of a nonlinear system at an equilibrium is asymptotically stable, then the equilibrium is **locally** asymptotically stable for the nonlinear system.

Let us apply this approach to the error dynamics of the unicycle. In the form we derived, the origin is **not** an unforced equilibrium: for $\mathbf{e} = \mathbf{0}$ the terms $v_d - v$ and $\omega_d - \omega$ must vanish to have $\dot{\mathbf{e}} = \mathbf{0}$. This is easily rectified by the following (invertible) input transformation:

$$w_1 = v_d \cos e_3 - v \tag{2.13}$$

$$w_2 = \omega_d - \omega \tag{2.14}$$

and, by substituting $\omega = \omega_d - w_2$, the error dynamics $\dot{\mathbf{e}} = \boldsymbol{\eta}(\mathbf{e}, \mathbf{w})$ takes the form

$$\dot{\mathbf{e}} = \underbrace{\begin{pmatrix} \omega_d e_2 \\ -\omega_d e_1 + v_d \sin e_3 \\ 0 \end{pmatrix}}_{f(\mathbf{e})} + \underbrace{\begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix}}_{G(\mathbf{e})} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \tag{2.15}$$

[1] Its eigenvalues have strictly negative real part.

which is at rest when $\mathbf{e} = \mathbf{0}, \mathbf{w} = \mathbf{0}$, i.e. $\boldsymbol{\eta}(\mathbf{0}, \mathbf{0}) = \mathbf{0}$. Both the drift and the input term are nonlinear, and in general the drift is also time-varying. The Jacobian matrices are

$$\frac{\partial \boldsymbol{\eta}}{\partial \mathbf{e}} = \begin{pmatrix} 0 & \omega_d - w_2 & 0 \\ -\omega_d + w_2 & 0 & v_d \cos e_3 \\ 0 & 0 & 0 \end{pmatrix} \qquad \frac{\partial \boldsymbol{\eta}}{\partial \mathbf{w}} = \begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix} \tag{2.16}$$

and by letting $\mathbf{e} = \mathbf{0}, \mathbf{w} = \mathbf{0}$ we obtain

$$A(t) = \begin{pmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & 0 & 0 \end{pmatrix} \qquad B = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}. \tag{2.17}$$

This system is controllable if and only if the matrix

$$\begin{pmatrix} B & AB & A^2B & \dots & A^{n-1}B \end{pmatrix} \tag{2.18}$$

have full rank, this can be seen by building just $\begin{pmatrix} B & AB \end{pmatrix}$:

$$\operatorname{rank} \begin{pmatrix} B & AB \end{pmatrix} = \operatorname{rank} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -\omega_d & v_d \\ 0 & 1 & 0 & 0 \end{pmatrix} = 3. \tag{2.19}$$

This holds if ω_d or v_d is non-zero, so the system lose controllability when the unicycle is stopping. Note that A is actually $A(t)$, due to the dependence of the reference inputs v_d and ω_d on time, hence the linear approximation is still a **time-varying** system. Let us define the linear feedback

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \underbrace{\begin{pmatrix} -k_1 & 0 & 0 \\ 0 & -k_2 & -k_3 \end{pmatrix}}_{K} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = K\mathbf{e} \tag{2.20}$$

the closed-loop error dynamics is

$$\dot{\mathbf{e}} = (A(t) + BK)\mathbf{e} = A_{cl}(t)\mathbf{e} = \begin{pmatrix} -k_1 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & -k_2 & -k_3 \end{pmatrix} \mathbf{e} \tag{2.21}$$

which is still time-varying. Letting

$$k_1 = k_3 = 2\zeta a \qquad k_2 = \frac{a^2 - \omega_d^2}{v_d} \tag{2.22}$$

with $a > 0$ and $\zeta \in (0, 1)$, the characteristic polynomial of $A_{cl}(t)$ becomes time-invariant and Hurwitz:

$$p(\lambda) = (\lambda + 2\zeta a)(\lambda^2 + 2\zeta a\lambda + a^2) \tag{2.23}$$

it has one real negative eigenvalue and two complex eigenvalues with damping ζ and natural frequency a.

Observation 2.1.1 : Caveats

Having the eigenvalues of $A_{cl}(t)$ fixed at every instant **does not guarantee** asymptotic stability of a time-varying system, unless v_d and ω_d are constant (rectilinear and circular trajectories). Even in this case, the asymptotic stability is only **local** (not global), since it comes from the indirect Lyapunov method. Moreover, $k_2 \to \infty$ as $v_d \to 0$, hence this controller can only be used with **persistent** Cartesian trajectories (stops are not allowed).

The **actual** velocity inputs v, ω are obtained by plugging the feedback controls w_1, w_2 in the input transformation:

$$v = v_d \cos e_3 - w_1 = v_d \cos e_3 + k_1 e_1 \tag{2.24}$$

$$\omega = \omega_d - w_2 = \omega_d + k_2 e_2 + k_3 e_3. \tag{2.25}$$

Note that $(v, \omega) \to (v_d, \omega_d)$ as $\mathbf{e} \to \mathbf{0}$, i.e. the control action reduces to the pure feedforward term. The controller is **static** and based on the state error, it needs v_d, ω_d (computed via flatness) and it needs θ both for the error rotation and for the input transformation. **Global stability** can be guaranteed by a nonlinear version of the feedback:

$$w_1 = -k_1(v_d, \omega_d)\, e_1 \tag{2.26}$$

$$w_2 = -k_2\, v_d \frac{\sin e_3}{e_3} e_2 - k_3(v_d, \omega_d)\, e_3 \tag{2.27}$$

with $k_2 > 0$ constant and k_1, k_3 bounded, positive and with bounded derivatives. The actual inputs are then recovered from the same input transformation.

2.1.2 Output Error Feedback

Another approach is to develop the feedback action from the **output (Cartesian) error** only, without computing a desired state trajectory $\mathbf{q}_d(t)$, using as **feedforward term** the velocity $\dot{\mathbf{p}}_d$ along the reference trajectory, where $\mathbf{p} = (x, y)^\top$ is the Cartesian position of the unicycle and $\mathbf{p}_d$ the desired one. We will design this controller through the technique of **exact input-output linearization**.

Exact Input-Output Linearization

Consider a driftless nonlinear system with the same number of inputs and outputs:

$$\dot{\mathbf{x}} = G(\mathbf{x})\mathbf{u} \qquad \mathbf{x} \in \mathbb{R}^n,\ \mathbf{u} \in \mathbb{R}^m,\ \mathbf{y} \in \mathbb{R}^m \tag{2.28}$$

$$\mathbf{y} = h(\mathbf{x}) \tag{2.29}$$

differentiating the output we obtain

$$\dot{\mathbf{y}} = \frac{\partial h}{\partial \mathbf{x}}\dot{\mathbf{x}} = \underbrace{\frac{\partial h}{\partial \mathbf{x}} G(\mathbf{x})}_{T(\mathbf{x})}\, \mathbf{u} = T(\mathbf{x})\mathbf{u} \tag{2.30}$$

where T is an $m \times m$ matrix called the **decoupling matrix**. If T is invertible, we can set the **linearizing feedback**

$$\mathbf{u} = T^{-1}(\mathbf{x})\mathbf{w} \implies \dot{\mathbf{y}} = T(\mathbf{x})T^{-1}(\mathbf{x})\mathbf{w} = \mathbf{w} \tag{2.31}$$

i.e. an **exactly linear** map between the new linearizing inputs $\mathbf{w}$ and (the time derivative of) the outputs: under the action of the linearizing feedback the system behaves as a simple (vector) integrator from $\mathbf{w}$ to $\mathbf{y}$.

Given a reference output $\mathbf{y}_d(t)$, the dynamics of the output error $\mathbf{e} = \mathbf{y}_d - \mathbf{y}$ is

$$\dot{\mathbf{e}} = \dot{\mathbf{y}}_d - \dot{\mathbf{y}} = \dot{\mathbf{y}}_d - \mathbf{w}. \tag{2.32}$$

Let $\mathbf{w} = \dot{\mathbf{y}}_d + K\mathbf{e}$ (feedforward plus proportional feedback), to obtain

$$\dot{\mathbf{e}} = -K\mathbf{e} \tag{2.33}$$

which is **globally exponentially stable** provided that the eigenvalues of K lie in the right half-plane. The final control law is

$$\mathbf{u} = T^{-1}(\mathbf{x})(\dot{\mathbf{y}}_d + K\mathbf{e}). \tag{2.34}$$

2.1.3 Static Feedback

Let us apply the exact input-output linearization to the unicycle, with output $\mathbf{p} = (x, y)^\top$. However, the decoupling matrix associated to the Cartesian position turns out to be **singular**:

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix} \tag{2.35}$$

hence exact input-output linearization is **not possible** for the output (x, y). The solution is to change slightly the output so that the new input-output map is invertible: we displace the output from the contact point of the wheel to a point B along the sagittal axis, at distance b:

$$\begin{pmatrix} x_B \\ y_B \end{pmatrix} = \begin{pmatrix} x + b\cos\theta \\ y + b\sin\theta \end{pmatrix}. \tag{2.36}$$

Differentiating with respect to time:

$$\begin{pmatrix} \dot{x}_B \\ \dot{y}_B \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -b\sin\theta \\ \sin\theta & b\cos\theta \end{pmatrix}}_{T(\theta),\ \det T = b} \begin{pmatrix} v \\ \omega \end{pmatrix} = T(\theta) \begin{pmatrix} v \\ \omega \end{pmatrix}. \tag{2.37}$$

If $b \neq 0$ we may set

$$\begin{pmatrix} v \\ \omega \end{pmatrix} = T^{-1}(\theta) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\dfrac{\sin\theta}{b} & \dfrac{\cos\theta}{b} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \tag{2.38}$$

obtaining an input-output linearized system:

$$\dot{x}_B = w_1 \tag{2.39}$$
$$\dot{y}_B = w_2 \tag{2.40}$$
$$\dot{\theta} = \frac{w_2\cos\theta - w_1\sin\theta}{b}. \tag{2.41}$$

We achieve **global exponential convergence** of x_B, y_B to the desired trajectory by letting

$$w_1 = \dot{x}_d + k_x(x_d - x_B) \tag{2.42}$$
$$w_2 = \dot{y}_d + k_y(y_d - y_B) \tag{2.43}$$

with $k_x, k_y > 0$. The orientation θ is **not** controlled by this scheme, which is based on output error feedback: there is a **zero dynamics** (the last equation above). Still, θ must evolve as dictated by flatness. An advantage of this approach is that the desired trajectory for B can be **arbitrary**, in particular square corners may be included. The controller is **static**, based on the output error, it needs $\dot{\mathbf{p}}_d$, and it needs x, y, θ for the output reconstruction and θ also for the input transformation.

2.1.4 Dynamic Feedback

Rather than displacing the controlled output to B, we can keep the (flat) output (x, y) and achieve exact linearization by using a **dynamic compensator**. To do this, we perform a **dynamic extension** on the driving velocity channel (we "slow down" the input), by considering the velocity v as a state of the system and its acceleration a as a new input:

$$\dot{v} = a. \tag{2.44}$$

We can now differentiate further the output:

$$\ddot{x} = a\cos\theta - v\sin\theta\,\omega \tag{2.45}$$
$$\ddot{y} = a\sin\theta + v\cos\theta\,\omega \tag{2.46}$$

the new decoupling matrix is

$$\begin{pmatrix} \ddot{x} \\ \ddot{y} \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -v\sin\theta \\ \sin\theta & v\cos\theta \end{pmatrix}}_{T(v,\theta),\ \det T = v} \begin{pmatrix} a \\ \omega \end{pmatrix} = T(v,\theta) \begin{pmatrix} a \\ \omega \end{pmatrix} \tag{2.47}$$

which is nonsingular if $v \neq 0$. Under this assumption we may set

$$\begin{pmatrix} a \\ \omega \end{pmatrix} = T^{-1}(v,\theta) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\dfrac{\sin\theta}{v} & \dfrac{\cos\theta}{v} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \tag{2.48}$$

obtaining a fully linearized system $\ddot{x} = w_1$, $\ddot{y} = w_2$. We achieve **global exponential convergence** of x, y to the desired trajectory by letting

$$w_1 = \ddot{x}_d + k_{p,x}(x_d - x) + k_{d,x}(\dot{x}_d - \dot{x}) \tag{2.49}$$

$$w_2 = \ddot{y}_d + k_{p,y}(y_d - y) + k_{d,y}(\dot{y}_d - \dot{y}) \tag{2.50}$$

with $k_{p,x}, k_{p,y}, k_{d,x}, k_{d,y} > 0$. Since we are controlling the original flat output, there is **no zero dynamics** with this approach. The desired trajectory must be twice differentiable and **persistent**, i.e. it must be $v_d \neq 0$ always (this singularity is structural in nonholonomic systems). The controller is **dynamic** (the velocity v is the state of the compensator, obtained by integrating a), based on the output error, it needs $\dot{\mathbf{p}}_d$ and $\ddot{\mathbf{p}}_d$, and it needs x, y for the error computation and θ for the input transformation.

2.1.5 Comparison of the Controllers

The three controllers have been compared in simulation while tracking a circle, a square and a figure 8, with the following outcomes.

- **Approximate linearization**: only **local** stability is guaranteed (and only for constant v_d, ω_d). When tracking a circle it converges to zero error; when tracking a square a new transient appears at each corner; for the figure 8 local stability is not guaranteed, but the observed performance is good.
- **Static input-output linearization**: the steady-state error norm is equal to b, since it is the point B that converges to the desired trajectory, while the contact point of the wheel stays at distance b from it. The displaced output provides a **lookahead** behavior when tracking a square. Reducing b reduces the steady-state error, but the steering velocity increases (the term $1/b$ appears in T^{-1}).
- **Dynamic input-output linearization**: zero steady-state error, reasonable velocities and faster transients (e.g. at the corners of the square).